Milvus: A Purpose-Built Vector Data Management System

Jianguo Wang*, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, Charles Xie
* Zilliz & Purdue University Zilliz
* *csjgwang@{zilliz.com; purdue.edu}* *{firstname.lastname}@zilliz.com*

ABSTRACT

Recently, there has been a pressing need to manage high-dimensional vector data in data science and AI applications. This trend is fueled by the proliferation of unstructured data and machine learning (ML), where ML models usually transform unstructured data into feature vectors for data analytics, e.g., product recommendation. Existing systems and algorithms for managing vector data have two limitations: (1) They incur serious performance issue when handling large-scale and dynamic vector data; and (2) They provide limited functionalities that cannot meet the requirements of versatile applications.

This paper presents Milvus, a purpose-built data management system to efficiently manage large-scale vector data. Milvus supports easy-to-use application interfaces (including SDKs and RESTful APIs); optimizes for the heterogeneous computing platform with modern CPUs and GPUs; enables advanced query processing beyond simple vector similarity search; handles dynamic data for fast updates while ensuring efficient query processing; and distributes data across multiple nodes to achieve scalability and availability. We first describe the design and implementation of Milvus. Then we demonstrate the real-world use cases supported by Milvus. In particular, we build a series of **10** applications (e.g., image/video search, chemical structure analysis, COVID-19 dataset search, personalized recommendation, biological multi-factor authentication, intelligent question answering) on top of Milvus. Finally, we experimentally evaluate Milvus with a wide range of systems including two open source systems (Vearch and Microsoft SPTAG) and three commercial systems. Experiments show that Milvus is up to two orders of magnitude faster than the competitors while providing more functionalities. Now Milvus is deployed by hundreds of organizations worldwide and it is also recognized as an incubation-stage project of the LF AI & Data Foundation. Milvus is open-sourced at *https://github.com/milvus-io/milvus.*

CCS CONCEPTS

• **Information systems → Database management system engines**; **Data access methods**;

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD '21, June 20–25, 2021, Virtual Event, China
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8343-1/21/06…$15.00
https://doi.org/10.1145/3448016.3457550

KEYWORDS

Vector database; High-dimensional similarity search; Heterogeneous computing; Data science; Machine learning

ACM Reference Format:
Jianguo Wang*, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21), June 20–25, 2021, Virtual Event, China.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3448016.3457550

1 INTRODUCTION

At Zilliz, we have experienced a growing need from various customers to manage large-scale high-dimensional vector data (ranging from 10s to 1000s of dimensions) in many data science and AI applications. This is largely due to two trends. The first one is an explosive growth of unstructured data such as images, videos, texts, medical data, and housing data due to the prevalence of smartphones, IoT devices, and social media apps. According to IDC, 80% of data will be unstructured by 2025 [36]. The second trend is the rapid development of machine learning that can effectively transform unstructured data into learned feature vectors for data analytics. In particular, a recent popular approach in recommender systems is called *vector embedding* that converts an item to a feature vector (such as item2vec [11], word2vec [52], doc2vec [37], graph2vec [26]) and provides recommendations via finding similar vectors [13, 15, 25, 51]. For example, YouTube embeds videos to vectors [15]; Airbnb models houses with vectors [25]; Bioscientists describe the molecular structural information of drug compounds using vectors [13, 51]. Besides that, images and texts are also naturally represented by vectors [8, 53].

Those applications present unique requirements and challenges for designing a scalable vector data management system. These include: (1) The need to support not only fast query processing on large-scale vector data but also the efficient handling of dynamic vector data (such as insertions and deletions). As an example, Youtube uploads 500 hours of user-generated videos per minute and meanwhile offers real-time recommendations [67]. (2) The need to provide advanced query processing such as attribute filtering [65] and multi-vector query processing [10] beyond simple vector similarity search. Here attribute filtering is to only search vectors that satisfy a given filtering condition, which is useful in e-commerce applications [65], e.g., finding the T-shirts similar to a given image vector that also cost less than $100. And multi-vector query processing targets for the scenario where each object is described by multiple vectors, e.g., profiling a person using a face vector and a posture vector in many computer vision applications [10, 56].

Milvus：专用向量数据管理系统

王建国*、易小萌、郭仁通、金海、徐鹏、李胜军、王翔宇、郭向洲、李成明、徐小海、于坤、袁雨星、邹英豪、龙继泉、蔡宇东、李振翔、张志峰、莫一华、顾军、姜瑞义、魏毅、Charles Xie * Zilliz
与普渡大学 Zilliz *csjgwang@{zilliz.com; purdue.edu} {firstname.lastname}@zilliz.com*

摘要

近年来，数据科学和人工智能应用中对高维向量数据的管理需求日益迫切。 1 这一趋势由非结构化数据的激增以及机器学习（ML）的发展所推动：ML模型通常将非结构化数据转换为特征向量，以支持数据分析任务，例如商品推荐。当前用于管理向量数据的系统与算法存在两大局限性：（1）在处理大规模动态向量数据时，性能问题严重；（2）功能有限，难以满足多样化应用场景的需求。本文提出Milvus——一款专为高效管理大规模向量数据而设计的数据管理系统。Milvus支持易用的应用程序接口（包括SDK和RESTful API）；针对现代CPU与GPU构成的异构计算平台进行优化；支持超越简单向量相似性搜索的高级查询处理；可高效处理动态数据并实现快速更新，同时保障查询处理效率；并通过跨多节点分布数据来实现可扩展性与高可用性。我们首先介绍Milvus的设计与实现；随后展示Milvus所支持的实际应用场景。具体而言，我们在Milvus之上构建了一系列**10** 个应用（例如图像/视频搜索、化学结构分析、COVID-19数据集搜索、个性化推荐、生物多因子认证、智能问答）。最后，我们通过一系列系统对Milvus展开实验评估，涵盖两个开源系统（Vearch与Microsoft SPTAG）及三个商用系统。实验结果表明，Milvus的性能最高可达竞品的两个数量级，并提供更丰富的功能。目前，Milvus已在全球数百家组织中部署，并被LF AI & Data基金会列为孵化阶段项目。Milvus已在*https://github.com/milvus-io/milvus*开源。

CCS 概念

• **信息系统 → 数据库管理系统 en-吉尼斯**；**数据访问方法**；

允许为个人或课堂教学目的，免费制作本作品全部或部分内容的数字版或印刷版副本，前提是不得将副本用于盈利或商业优势之目的，且所有副本须在首页注明本声明及完整引用信息。本作品中由美国计算机协会（ACM）以外机构所拥有的各组成部分的版权必须予以尊重。允许在注明出处的前提下进行摘要编撰。如需以其他方式复制、重新出版、上传至服务器，或向邮件列表分发，则须事先获得特定授权和/或支付相应费用。授权申请请发送至 permissions@acm.org。*SIGMOD '21，2021年6月20–25日*，中国虚拟会议© 2021 美国计算机协会。ACM ISBN 978-1-4503-8343-1/21/06···$15.00 https://doi.org/10.1145/3448016.3457550

关键词

向量数据库；高维相似性搜索；异构计算；数据科学；机器学习

ACM Reference Format:
王建国*，易小萌，郭仁通，金海，徐鹏，李胜军，
王翔宇、郭向洲、李成明、徐小海、于坤、袁雨星、邹英豪、龙继泉、蔡宇东、李振翔、张志峰，
莫一华、顾军、姜瑞义、魏毅、Charles Xie . 2021. Milvus：一款面向特定用途的 - 构建的向量数据管理系统。《*2021*年国际数据管理会议论文集》（*SIGMOD '21*），*2021*年*6*月*20–25*日，中国虚拟会议。ACM，美国纽约州纽约市，14页。https://doi.org/10.1145/3448016.3457550

1 引言

在 Zilliz，我们切身感受到来自各类客户日益增长的需求：在众多数据科学与人工智能应用中，高效管理大规模高维向量数据（维度范围从数十维到数千维）。这一趋势主要源于两大动因。其一是非结构化数据（如图像、视频、文本、医疗数据及住房数据）的爆发式增长，这得益于智能手机、物联网设备及社交媒体应用的普及。据 IDC 预测，到 2025 [36]，全球数据中将有 80% 属于非结构化数据。其二是机器学习技术的快速发展，该技术可高效地将非结构化数据转化为用于数据分析的特征向量。尤其在推荐系统领域，一种近期广为流行的方法称为向量嵌入，它将某一物品映射为一个特征向量（例如 item2vec [11], 、word2vec [52], 、doc2vec [37],、graph2vec [26]），并通过查找相似向量 [13, 15, 25, 51]来提供推荐服务。例如，YouTube 将视频嵌入为向量 [15]；Airbnb 使用向量对房源进行建模 [25]；生物科学家则利用向量描述药物化合物的分子结构信息 [13, 51]。此外，图像和文本也天然适合以向量形式表示 [8, 53]。

这些应用对设计可扩展的向量数据管理系统提出了独特的需求与挑战，主要包括：（1）不仅需要支持大规模向量数据的快速查询处理，还需高效处理动态向量数据（例如插入和删除操作）。以 YouTube 为例，其每分钟上传 500 小时的用户生成视频，同时提供实时推荐 [67]。（2）需要提供高级查询处理能力，例如属性过滤 [65] 和多向量查询处理 [10] ，而不仅限于简单的向量相似性搜索。其中，属性过滤是指仅检索满足特定过滤条件的向量，这在电子商务应用 [65]，中十分有用，例如：在给定图像向量的基础上，查找与其相似且价格低于 100 美元的 T 恤；而多向量查询处理则面向每个对象由多个向量描述的场景，例如在诸多计算机视觉应用 [10, 56] 中，使用人脸向量和姿态向量共同刻画一个人物。

Table 1: System comparison

	Billion-Scale Data	Dynamic Data	GPU	Attribute Filtering	Multi-Vector Query	Distributed System
Facebook Faiss [3, 35]	✓	✗	✓	✗	✗	✗
Microsoft SPTAG [14]	✓	✗	✗	✗	✗	✗
ElasticSearch [2]	✗	✓	✗	✓	✗	✓
Jingdong Vearch [4, 39]	✗	✓	✓	✓	✗	✓
Alibaba AnalyticDB-V [65]	✓	✓	✗	✓	✗	✓
Alibaba PASE (PostgreSQL) [68]	✗	✓	✗	✓	✗	✗
Milvus (this paper)	✓	✓	✓	✓	✓	✓

Existing works on vector data management mainly focus on vector similarity search [14, 20, 22, 33, 35, 39, 45, 46, 48, 49, 57, 65, 68], but they cannot meet the above requirements due to poor performance (on large-scale and dynamic vector data) and limited functionalities (e.g., not being capable of supporting attribute filtering and multi-vector queries) to support versatile data science and AI applications.

More specifically, we classify existing works into two categories: *algorithms* and *systems*. For the algorithmic works on vector similarity search, e.g., [20, 22, 33, 45, 46, 48, 49, 57], together with their open-source implementation libraries (exemplified by Facebook Faiss [35] and Microsoft SPTAG [14]), there are several limitations. (1) They are algorithms and libraries, not a full-fledged system that manages vector data. They cannot handle large amount of data very well since they assume that all the data and index are stored in main memory and cannot span multiple machines. (2) Those works usually assume data to be static once ingested into the system and cannot easily handle dynamic data while ensuring fast real-time searches. (3) They do not support advanced query processing. (4) Those works are not optimized for the heterogeneous computing architecture with CPUs and GPUs.

For the system works on vector similarity search, e.g., Alibaba AnalyticDB-V [65] and Alibaba PASE (PostgreSQL) [68], they follow the one-size-fits-all approach to extend relational databases for supporting vector data by adding a table column called "vector column" to store vectors. However, those systems are not specialized for managing vector data and they do not treat vectors as first-class citizens. (1) Legacy database components such as optimizer and storage engine prevent fine-tuned optimizations for vectors, e.g., the query optimizer misses significant opportunity to best leverage CPU and GPU for processing vector data. (2) They do not support advanced query processing such as multi-vector queries.

Another relevant system is Vearch [4, 39], which is designed for vector search. But Vearch is not efficient on large-scale data. Experiments (Figure 8 and Figure 15) show that Milvus, the system introduced in this paper, is 6.4× ∼ 47.0× faster than Vearch. Also, Vearch does not support multi-vector query processing.

This paper presents Milvus, a purpose-built data management system to efficiently store and search large-scale vector data for data science and AI applications. It is a specialized system for high-dimensional vectors following the design practice of one-size-not-fits-all [60] in contrast to generalizing relational databases to support vectors. Milvus provides many application interfaces (including SDKs in Python/Java/Go/C++ and RESTful APIs) that can be easily used by applications. Milvus is highly tuned for the heterogeneous computing architecture with modern CPUs and GPUs (multiple GPU devices) for the best efficiency. It supports versatile query types such as vector similarity search with various similarity functions, attribute filtering, and multi-vector query processing. It provides different types of indexes (e.g., quantization-based indexes [33, 35] and graph-based indexes [20, 49]) and develops an extensible interface to easily incorporate new indexes into the system. Milvus manages dynamic vector data (e.g., insertions and deletions) via an LSM-based structure while providing consistent real-time searches with snapshot isolation. Milvus is also a distributed data management system deployed across multiple nodes to achieve scalability and availability. Table 1 highlights the main differences between Milvus and other systems.

In terms of implementation, Milvus is built on top of Facebook Faiss [3, 35], an open-source C++ library for vector similarity search. But Milvus significantly enhances Faiss with improved performance (e.g., optimizing for the heterogeneous computing platform in Sec. 3, supporting dynamic data management efficiently in Sec. 2.3 and distributed query processing in Sec. 5.3), enhanced functionalities (e.g., attribute filtering and multi-vector query processing in Sec. 4), and better usability (e.g., application interfaces in Sec. 2.1) to be a full-fledged easy-to-use vector data management system.

Product impact. Milvus is adopted by hundreds of organizations and institutions worldwide in various fields such as image processing, computer vision, natural language processing, voice recognition, recommender systems, and drug discovery. More importantly, Milvus was accepted as an incubation-stage project of the LF AI & Data Foundation in January 2020.[1]

Contributions. This paper makes the following contributions:

- **System design and implementation** (Sec. 2 and Sec. 5): The overall contribution is the design and implementation of Milvus, a purpose-built vector data management system for managing large-scale and dynamic vector data to enable data science and AI applications. Milvus is open-sourced at *https://github.com/milvus-io/milvus*.
- **Heterogeneous computing** (Sec. 3): We optimize Milvus for the heterogeneous hardware platform with modern CPUs and GPUs for fast query processing. For CPU-oriented design, we propose both cache-aware and SIMD-aware (e.g., SSE, AVX, AVX2, AVX512) optimizations. For GPU-oriented design, we design a new hybrid index that takes advantages of the best of CPU and GPU, and we also develop a new scheduling strategy to support multiple GPU devices.
- **Advanced query processing** (Sec. 4): We support attribute filtering and multi-vector query processing beyond simple vector similarity search in Milvus. In particular, we design a new partition-based algorithm for attribute filtering and two algorithms (vector fusion and iterative merging) for multi-vector query processing.

[1] https://lfaidata.foundation/projects/milvus

表 1：系统对比

	十亿级数据	动态数据	GPU	属性过滤	多向量查询	分布式系统
Facebook Faiss [3, 35]	✓	✗	✓	✗	✗	✗
Microsoft SPTAG [14]	✓	✗	✗	✗	✗	✗
ElasticSearch [2]	✗	✓	✗	✓	✗	✓
京东 Vearch [4, 39]	✗	✓	✓	✓	✗	✓
阿里巴巴 AnalyticDB-V [65]	✓	✓	✗	✓	✗	✓
Alibaba PASE（PostgreSQL） [68]	✗	✓	✗	✓	✗	✗
Milvus（本文）	✓	✓	✓	✓	✓	✓

现有向量数据管理研究主要聚焦于向量相似性搜索 [14, 20, 22, 33, 35, 39, 45, 46, 48, 49, 57, 65, 68],，但由于在大规模及动态向量数据上的性能较差，且功能有限（例如无法支持属性过滤和多向量查询），因而难以满足上述需求，亦无法充分支撑多样化的数据科学与人工智能应用。

更具体而言，我们将现有工作分为两类：算法和系统。对于面向向量相似性搜索的算法类工作（例如 [20, 22, 33, 45, 46, 48, 49, 57], 及其开源实现库（以 Facebook Faiss [35] 和 Microsoft SPTAG [14] 为代表）），存在若干局限性：（1）它们属于算法和库，而非管理向量数据的完整系统；由于假设所有数据及索引均驻留在主存中且无法跨多台机器部署，因此难以高效处理海量数据。（2）此类工作通常假设数据一旦被摄入系统即为静态，难以在保障实时快速搜索的前提下灵活应对动态数据。（3）它们不支持高级查询处理。（4）这些工作并未针对配备 CPU 和 GPU 的异构计算架构进行优化。

该系统面向向量相似性搜索，例如阿里巴巴 AnalyticDB-V [65] 和 Alibaba PASE（PostgreSQL） [68], ，均采用"一刀切"方法，通过在关系型数据库中新增一个名为"向量列"的表列来存储向量，从而扩展其对向量数据的支持能力。然而，这些系统并非专为管理向量数据而设计，也未将向量视为头等公民。（1）传统数据库组件（如查询优化器和存储引擎）阻碍了针对向量的精细化优化；例如，查询优化器未能充分把握利用 CPU 和 GPU 高效处理向量数据的重大机会。（2）它们不支持多向量查询等高级查询处理功能。

另一相关系统是 Vearch [4, 39], ，其专为向量搜索而设计。但 Vearch 在大规模数据场景下效率较低。实验结果（图 8 和图 15 ）表明，本文所介绍的系统 Milvus 比 Vearch 快 6.4× ∼ 47.0× 倍。此外，Vearch 不支持多向量查询处理。

本文介绍了 Milvus——一种专为高效存储与检索大规模向量数据而设计的数据管理系统，适用于数据科学和人工智能应用。它是一种面向高维向量的专用系统，遵循"一刀切不适用"的设计理念 [60] ，与将关系型数据库泛化以支持向量的做法截然不同。Milvus 提供了多种应用程序接口（包括 Python/Java/Go/C++ 语言的 SDK 以及 RESTful API），便于各类应用便捷调用。Milvus 针对现代 CPU 和 GPU（含多 GPU 设备）构成的异构计算架构进行了深度优化，以实现最佳性能。它支持多种查询类型，例如基于多种相似度函数的向量相似性搜索、属性过滤以及多向量查询处理。Milvus 提供了多种类型的索引（例如基于量化的索引 [33, 35]和基于图的索引 [20, 49]），并设计了可扩展的接口，以便轻松将新型索引集成至系统中。Milvus 采用基于 LSM 的结构管理动态向量数据（如插入与删除操作），同时借助快照隔离机制保障一致的实时查询能力。此外，Milvus 还是一个分布式数据管理系统，可跨多个节点部署，从而实现可扩展性与高可用性。表 1 概括了 Milvus 与其他系统的主要差异。

在实现层面，Milvus 构建于开源向量相似性搜索 C++ 库 Facebook Faiss [3, 35], 之上。但 Milvus 在 Faiss 基础上进行了显著增强：性能方面（例如，第 3 节中针对异构计算平台的优化、第 2.3 节中对动态数据管理的高效支持，以及第 5.3 节中的分布式查询处理）；功能方面（例如，第 4 节中的属性过滤与多向量查询处理）；以及易用性方面（例如，第 2.1 节中的应用接口），从而发展成为一个功能完备、易于使用的专用向量数据管理系统。

产品影响力。Milvus 已被全球数百家组织和机构广泛应用于图像处理、计算机视觉、自然语言处理、语音识别、推荐系统及药物发现等多个领域。更重要的是，Milvus 于 2020 年 1 月正式成为 LF AI & Data 基金会的孵化阶段项目。[1]

贡献。本文的主要贡献如下：

- **系统设计与实现**（第 2 节和第 5 节）：本文的核心贡献在于设计并实现了 Milvus——一个专为管理大规模动态向量数据而打造的专用向量数据管理系统，旨在赋能数据科学与人工智能应用。Milvus 已在 *https://github.com/milvus-io/milvus* 开源。
- **异构计算**（第 3 节）：我们针对配备现代 CPU 和 GPU 的异构硬件平台优化 Milvus，以实现快速查询处理。在面向 CPU 的设计中，我们提出了缓存感知和 SIMD 感知（例如 SSE、AVX、AVX2、AVX512）两类优化方案。在面向 GPU 的设计中，我们设计了一种新型混合索引，以充分发挥 CPU 和 GPU 各自的优势；同时，我们还开发了一种新型调度策略，以支持多 GPU 设备。
- **高级查询处理**（第 4 节）：Milvus 不仅支持简单的向量相似性搜索，还支持属性过滤和多向量查询处理。具体而言，我们为属性过滤设计了一种基于分区的算法，并为多向量查询处理设计了两种算法（向量融合与迭代合并）。

[1] https://lfaidata.foundation/projects/milvus

- **Novel applications** (Sec. 6): We describe novel applications powered by Milvus. In particular, we build a series of **10** applications[2] on top of Milvus to demonstrate its broad applicability including image search, video search, chemical structure analysis, COVID-19 dataset search, personalized recommendation, biological multi-factor authentication, intelligent question answering, image-text retrieval, cross-modal pedestrian search, and recipe-food search.

2 SYSTEM DESIGN

In this section, we present an overview of Milvus. Figure 1 shows the architecture of Milvus with three major components: query engine, GPU engine, and storage engine. The query engine supports efficient query processing over vector data and it is optimized for modern CPUs by reducing cache misses and leveraging SIMD instructions. The GPU engine is a co-processing engine that accelerates performance with vast parallelism. It also supports multiple GPU devices for efficiency. The storage engine enables data durability and incorporates an LSM-based structure for dynamic data management. It runs on various file systems (including local file systems, Amazon S3, and HDFS) with a bufferpool in memory.

2.1 Query Processing

We first present the concept of entity used in Milvus and then explain query types, similarity functions, and application interfaces.

Entity. To best capture versatile data science and AI applications, Milvus supports query processing over both vector data and non-vector data. We define the term *entity* as follows to incorporate the two. Each entity in Milvus is described as one or more vectors and optionally some numerical attributes (non-vector data). For example, in the image search application, the numerical attributes can represent the age and height of a person in addition to possibly multiple machine-learned feature vectors of his/her photos (e.g., describing front-face, side-face, or posture [10]). In the current version of Milvus, we only support numerical attributes as observed from many applications. But in the future, we plan to support categorical attributes with indexes like inverted lists or bitmaps [64].

Query types. Milvus supports three primitive query types:

- Vector query: This query type is the traditional vector similarity search [33, 41, 48, 49], where each entity is described as a single vector. The system returns k most similar vectors where k is a user-input parameter.
- Attribute filtering: Each entity is specified by a single vector and some attributes [65]. The system returns k most similar vectors while adhering to the attributes constraints. As an example in recommender systems, users want to find similar clothes to a given query image while the price is below $100.
- Multi-vector query: Each entity is stored as multiple vectors [10]. The query returns top-k similar entities according to an aggregation function (e.g., weighted sum) between multiple vectors.

Similarity functions. Milvus offers commonly used similarity metrics, including Euclidean distance, inner product, cosine similarity, Hamming distance, and Jaccard distance, allowing applications to explore vector similarity in the most effective approach.

[2]https://github.com/milvus-io/bootcamp/tree/master/EN_solutions

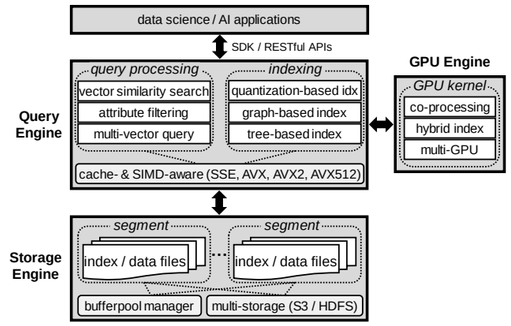


Figure 1: System architecture of Milvus

Application interfaces. Milvus provides easy-to-use SDK (software development kit) interfaces that can be directly called in applications written in various languages including Python, Java, Go, and C++. Milvus also supports RESTful APIs for web applications.

2.2 Indexing

Indexing is of tremendous importance to query processing in Milvus. But a challenging issue we face is to decide which indexes to support in Milvus, because there are numerous indexes developed for vector similarity search. The latest benchmark [41] shows that there is no winner in all scenarios and each index comes with tradeoffs in performance, accuracy, and space overhead.

In Milvus, we mainly support two types of indexes:[3] quantization-based indexes (including IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) and graph-based indexes (including HNSW [49] and RNSG [20]) to serve different applications. The design decision is based on factors including the latest literature review [41], industrial-strength systems (e.g., Alibaba PASE [68], Alibaba AnalyticDB-V [65], Jingdong Vearch [39]), open-source libraries (e.g., Facebook Faiss [3, 35]), and inputs from customers. We exclude LSH-based approaches because they have lower accuracy than quantization-based approaches on billion-scale data [65, 68].

Considering there are many new indexes coming out every year, Milvus is designed to easily incorporate the new indexes with a high-level abstraction. Developers only need to implement a few pre-defined interfaces for adding a new index. Our hope is that Milvus can eventually become a standard platform for vector data management with versatile indexes.

2.3 Dynamic Data Management

Milvus supports efficient insertions and deletions by adopting the idea of LSM-tree [47]. Newly inserted entities are stored in memory first as MemTable. Once the accumulated size reaches a threshold, or once every second, the MemTable becomes immutable and then gets flushed to disk as a new segment. Smaller segments are merged into larger ones for fast sequential access. Milvus implements a tiered merge policy (also used in Apache Lucene) that aims to merge segments of approximately equal sizes until a configurable size limit (e.g., 1GB) is reached. Deletions are supported in the same out-of-place approach except that the obsoleted vectors are

[3]Milvus also supports tree-based indexes, e.g., ANNOY [1].

- **新型应用**（第6节）：我们介绍了由Milvus驱动的新型应用。具体而言，我们在Milvus之上构建了一系列**10** 个[2]应用，以展示其广泛的适用性，包括图像搜索、视频搜索、化学结构分析、COVID-19数据集搜索、个性化推荐、生物多因素身份认证、智能问答、图文检索、跨模态行人搜索以及食谱-食物搜索。

2 系统设计

本节将概述 Milvus 的整体架构。图 1 展示了 Milvus 的架构，其包含三大核心组件：查询引擎、GPU 引擎和存储引擎。查询引擎支持对向量数据的高效查询处理，并针对现代 CPU 进行了深度优化，通过减少缓存缺失并充分利用 SIMD 指令来提升性能。GPU 引擎是一种协同处理引擎，借助大规模并行计算能力显著加速性能，同时支持多 GPU 设备以进一步提升效率。存储引擎保障数据持久性，并采用基于 LSM 的结构实现动态数据管理；它可运行于多种文件系统之上（包括本地文件系统、Amazon S3 和 HDFS），并在内存中配备缓冲池。

2.1 查询处理

我们首先介绍 Milvus 中所使用的实体概念，然后阐述查询类型、相似度函数以及应用接口。

实体。为最大限度地支持多样化的数据科学与人工智能应用场景，Milvus 支持对向量数据和非向量数据进行查询处理。我们对术语实体定义如下，以涵盖这两类数据：Milvus 中的每个实体均由一个或多个向量描述，并可选地附带若干数值属性（即非向量数据）。例如，在图像搜索应用中，数值属性可用于表示某个人的年龄和身高，此外还可包含其多张照片所对应的若干机器学习特征向量（例如，分别描述正面、侧面或姿态 [10]）。在 Milvus 当前版本中，我们仅支持数值属性，这是基于大量实际应用观察所得；但未来我们计划支持类别属性，并为其构建倒排列表或位图 [64]等索引结构。

查询类型。Milvus 支持三种基本查询类型：

- 向量查询：此类查询为传统的向量相似性搜索 [33, 41, 48, 49]，，其中每个实体均以单个向量表示。系统返回 k 最相似的向量，其中 k 为用户输入的参数。
- 属性过滤：每个实体由一个向量及若干属性 [65]共同描述。系统在满足属性约束的前提下，返回 k 最相似的向量。例如在推荐系统中，用户希望基于给定的查询图像查找相似服装，同时要求价格低于100美元。
- 多向量查询：每个实体以多个向量 [10]形式存储。该查询根据多个向量间的聚合函数（例如加权和）返回最相似的前k 个实体。

据聚合函数（例如加权和）在多个向量之间进行计算。

相似度函数。Milvus 提供了常用的相似度度量方法，包括欧氏距离、内积、余弦相似度、汉明距离和 Jaccard 距离，使应用程序能够以最有效的方式探索向量相似性。

[2]https://github.com/milvus-io/bootcamp/tree/master/EN_解决方案

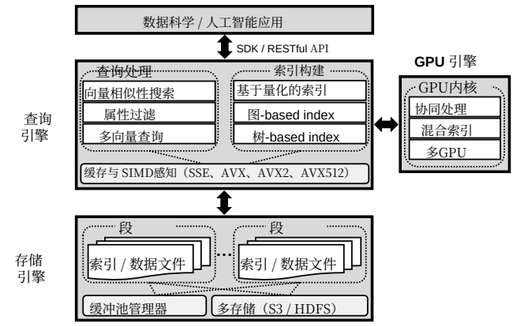


图1：Milvus 系统架构

应用接口。Milvus 提供了易于使用的 SDK（软件开发工具包）接口，可直接在使用 Python、Java、Go 和 C++ 等多种语言编写的应用程序中调用。Milvus 还支持面向 Web 应用的 RESTful API。

2.2 索引构建

索引构建对 Milvus 中的查询处理至关重要。但我们面临的一个挑战性问题是：如何确定 Milvus 应支持哪些索引，因为目前已有大量为向量相似性搜索而开发的索引方法。最新基准测试 [41]表明，在所有场景中并不存在一种 “ 通吃型 ” 最优索引，每种索引在性能、精度和空间开销方面均存在各自的权衡取舍。

在 Milvus 中，我们主要支持两类索引：[3]基于量化的索引（包括 IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], 和 IVF_PQ [3, 22, 33, 35]）以及基于图的索引（包括 HNSW [49] 和 RNSG [20]），以满足不同应用场景的需求。该设计决策综合考虑了多方面因素，包括最新文献综述 [41], 、工业级系统（例如 Alibaba PASE [68],、阿里巴巴 AnalyticDB-V [65], 、京东 Vearch [39]）、开源库（例如 Facebook Faiss [3, 35]）以及客户反馈。我们未采用基于局部敏感哈希的方法，因其在十亿级数据上的精度低于基于量化的方法 [65, 68]。

鉴于每年都有大量新型索引不断涌现，Milvus 在设计上采用了高层抽象机制，以便轻松集成各类新索引。开发者仅需实现若干预定义接口，即可添加一种新索引。我们期望 Milvus 最终能发展成为支持丰富多样的索引、面向向量数据管理的标准平台。

2.3 动态数据管理

Milvus 借鉴 LSM 树 [47] 的思想，支持高效的插入与删除操作。新插入的实体首先以 MemTable 形式暂存于内存中；当累积大小达到阈值，或每隔一秒，该 MemTable 即变为不可变状态，并被刷新至磁盘，形成一个新的段。较小的段会合并为较大的段，以实现快速的顺序访问。Milvus 实现了一种分层合并策略（该策略同样应用于 Apache Lucene），其目标是将尺寸大致相等的段进行合并，直至达到可配置的尺寸上限（例如 1 GB）。删除操作采用相同的原地外更新方式，区别仅在于已废弃的向量将在段合并过程中被移除。

[3]Milvus 还支持基于树的索引，例如 ANNOY [1]。

removed during segment merge. Updates are supported by deletions and insertions. By default, Milvus builds indexes only for large segments (e.g., > 1GB) but users are allowed to manually build indexes for segments of any size if necessary. Both index and data are stored in the same segment. Thus, the segment is the basic unit of searching, scheduling, and buffering.

Milvus offers snapshot isolation to make sure reads and writes share a consistent view and do not interfere with each other. We present the details of snapshot isolation in Sec. 5.2.

2.4 Storage Management

As mentioned in Sec. 2.1, each entity is expressed as one or more vectors and optionally some attributes. Thus, each entity can be regarded as a row in an entity table. To facilitate query processing, Milvus physically stores the entity table in a columnar fashion.

Vector storage. For single-vector entities, Milvus stores all the vectors continuously without explicitly storing the row IDs. In this way, all the vectors are sorted by row IDs. Given a row ID, Milvus can directly access the corresponding vector since each vector is of the same length. For multi-vector entities, Milvus stores the vectors of different entities in a columnar fashion. For example, assuming that there are three entities (A, B, and C) in the database and each entity has two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$, then all the $\mathbf{v}_1$ of different entities are stored together and all the $\mathbf{v}_2$ are stored together. That is, the storage format is $\{A.\mathbf{v}_1, B.\mathbf{v}_1, C.\mathbf{v}_1, A.\mathbf{v}_2, B.\mathbf{v}_2, C.\mathbf{v}_2\}$.

Attribute storage. The attributes are stored column by column. In particular, each attribute column is stored as an array of ⟨*key*,*value*⟩ pairs where the *key* is the attribute value and *value* is the row ID, sorted by the *key*. Besides that, we build skip pointers (i.e., min/max values) following Snowflake [16] as indexing for the data pages on disk. This allows efficient point query and range query in that column, e.g., price is less than $100.

Bufferpool. Milvus assumes that most (if not all) data and index are resident in memory for high performance. If not, it relies on an LRU-based buffer manager. In particular, the caching unit is a segment, which is the basic searching unit as explained in Sec. 2.3.

Multi-storage. For flexibility and reliability, Milvus supports multiple file systems including local file systems, Amazon S3, and HDFS for the underlying data storage. This also facilitates the deployment of Milvus in the cloud.

2.5 Heterogeneous Computing

Milvus is highly optimized for the heterogeneous computing platform that includes CPUs and GPUs. Sec. 3 presents the details.

2.6 Distributed System

Milvus can function as a distributed system deployed across multiple nodes. It adopts modern design practices in distributed systems and cloud systems such as storage/compute separation, shared storage, read/write separation, and single-writer-multi-reader. Sec. 5.3 explains more.

3 HETEROGENEOUS COMPUTING

In this section, we present the optimizations for Milvus to best leverage the heterogeneous computing platform involving both CPUs and GPUs to achieve high performance.

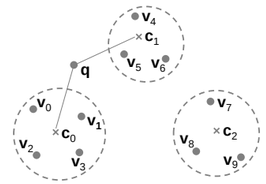


Figure 2: An example of quantization

As explained in Sec. 2.2, Milvus mainly supports quantization-based indexes (including IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) and graph-based indexes (including HNSW [49] and RNSG [20]). In this section, we use quantization-based indexes to illustrate our optimizations because they consume much less memory and are much faster to build index while achieving decent query performance when compared to graph-based indexes [65, 68]. Note that many optimizations (such as SIMD and GPU optimizations) can be applied to graph-based indexes.

3.1 Background

Before diving into optimizations, we explain vector quantization and quantization-based indexes. The main idea of vector quantization is to apply a quantizer z to map a vector $\mathbf{v}$ to a codeword $z(\mathbf{v})$ chosen from a codebook C [33]. The K-means clustering algorithm is commonly used to construct the codebook C where each codeword is the centroid and $z(\mathbf{v})$ is the closest centroid to $\mathbf{v}$. Figure 2 shows an example of 10 vectors ($\mathbf{v}_0$ to $\mathbf{v}_9$) of three clusters with centroids being $\mathbf{c}_0$ to $\mathbf{c}_2$, then $z(\mathbf{v}_0)$, $z(\mathbf{v}_1)$, $z(\mathbf{v}_2)$, or $z(\mathbf{v}_3)$ is $\mathbf{c}_0$.

Quantization-based indexes (such as IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) use two quantizers: coarse quantizer and fine quantizer. The coarse quantizer applies the K-means algorithm (e.g., K is 16384 in Milvus and Faiss [3]) to cluster vectors into K buckets. And the fine quantizer encodes the vectors within each bucket. Different indexes may use different fine quantizers. IVF_FLAT uses the original vector representation; IVF_SQ8 uses a compressed representation for the vectors by adopting one-dimensional quantizer (called "scalar quantizer") to compress a 4-byte float value to a 1-byte integer; and IVF_PQ uses product quantization that splits each vector into multiple sub-vectors and applies K-means for each sub-space.

Query processing (of a query $\mathbf{q}$) over quantization-based indexes takes two steps: (1) Find the closest n_{probe} buckets (or clusters) based on the distance between $\mathbf{q}$ and the centroid of each bucket. For example, assuming n_{probe} is 2 in Figure 2, then the closest two buckets of $\mathbf{q}$ are centered at $\mathbf{c}_0$ and $\mathbf{c}_1$. The parameter n_{probe} controls the tradeoff between accuracy and performance. Higher n_{probe} produces better accuracy but worse performance. (2) Search within each of the n_{probe} relevant buckets based on different fine quantizers. For example, if the index in Figure 2 is IVF_FLAT, then it needs to scan the vectors $\mathbf{v}_0$ to $\mathbf{v}_6$ in the two buckets.

3.2 CPU-oriented Optimizations

3.2.1 Cache-aware Optimizations in Milvus

The fundamental problem for query processing over quantization-based indexes is that, given a collection of m queries $\{\mathbf{q}_1, \mathbf{q}_2, ..., \mathbf{q}_m\}$ and a collection of n data vectors $\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n\}$, how to quickly

更新操作则通过先删除、再插入的方式实现。默认情况下，Milvus 仅对大型段（例如 ＞1GB）构建索引，但用户也可根据需要手动为任意大小的段构建索引。索引与数据均存储于同一段中，因此段是搜索、调度及缓冲的基本单位。

Milvus 提供快照隔离机制，以确保读操作与写操作共享一致的视图且互不干扰。我们将在第5.2节中详细介绍快照隔离机制。

2.4 存储管理

如第2.1节所述，每个实体以一个或多个向量及可选的若干属性来表示。因此，每个实体均可视为一张实体表中的一行。为便于查询处理，Milvus 在物理存储上采用列式方式存储实体表。

向量存储。对于单向量实体，Milvus 将所有向量连续存储，且不显式存储行 ID。这样，所有向量即按行 ID 排序。给定一个行 ID，Milvus 可直接访问对应向量，因为每个向量长度相同。对于多向量实体，Milvus 以列式方式存储不同实体的向量。例如，假设数据库中存在三个实体（A、B 和 C），且每个实体均包含两个向量 $\mathbf{v}_1$ 与 $\mathbf{v}_2$，则所有实体的 $\mathbf{v}_1$ 被集中存储，所有实体的 $\mathbf{v}_2$ 也被集中存储。即，其存储格式为 $\{A.\mathbf{v}_1, B.\mathbf{v}_1, C.\mathbf{v}_1, A.\mathbf{v}_2, B.\mathbf{v}_2, C.\mathbf{v}_2\}$。

属性存储。属性按列存储。具体而言，每个属性列均以 键,值 对数组形式存储，其中 键 为属性值，值 为行 ID，并按 键 排序。此外，我们参照 Snowflake [16] 的方式，在磁盘数据页上构建跳指针（即最小值/最大值）作为索引。这使得该列能够高效执行点查询和范围查询，例如“价格低于 100 美元”等条件查询。

缓冲池。Milvus 假设为实现高性能，大部分（甚至全部）数据和索引均驻留在内存中；若非如此，则依赖基于 LRU 的缓冲管理器。具体而言，缓存单元为“段”，即第 2.3 节所述的基本搜索单元。

多存储。为兼顾灵活性与可靠性，Milvus 支持多种文件系统作为底层数据存储，包括本地文件系统、AmazonS3 和 HDFS。这也便于 Milvus 在云环境中部署。

2.5 异构计算

Milvus 高度优化了对包含 CPU 和 GPU 的异构计算平台的支持。第 3 节将详细介绍相关内容。

2.6 分布式系统

Milvus 可作为分布式系统跨多个节点部署。它采用了分布式系统与云系统中的现代设计实践，例如存储计算分离、共享存储、读写分离以及单写多读。第 5.3 节将作进一步说明。

3 异构计算

本节介绍针对 Milvus 的优化方案，旨在充分利用包含 CPU 和 GPU 的异构计算平台，以实现高性能。

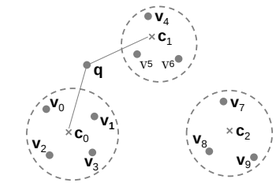


图2：量化的示例

如第 2.2 节所述，Milvus 主要支持基于量化的索引（包括 IVF_FLAT [3, 33, 35]，、IVF_SQ8 [3, 35]，和 IVF_PQ [3, 22, 33, 35]）以及基于图的索引（包括 HNSW [49] 和 RNSG [20]）。本节以基于量化的索引为例说明我们的优化方案，因为与基于图的索引 [65, 68] 相比，它们内存占用显著更低、建索引速度明显更快，同时仍能提供良好的查询性能。需要注意的是，许多优化（例如 SIMD 优化和 GPU 优化）同样可应用于基于图的索引。

3.1 背景

在深入探讨优化方法之前，我们先介绍向量量化和量化索引。向量量化的核心思想是应用一个量化器 z，将向量$\mathbf{v}$ 映射至码本C中选定的一个码字 $z(\mathbf{v})$。通常采用K-均值聚类算法构建码本C，其中每个码字即为一个簇的质心，而 $z(\mathbf{v})$表示距离$\mathbf{v}$最近的质心。图2展示了10个三维向量（$\mathbf{v}_0$ 至$\mathbf{v}_9$）划分为三个簇的示例，其质心分别为$\mathbf{c}_0$ 至$\mathbf{c}_2$；此时 $z(\mathbf{v}_0)$、$z(\mathbf{v}_1)$、$z(\mathbf{v}_2)$或 $z(\mathbf{v}_3)$均为$\mathbf{c}_0$。

量化索引（例如IVF_FLAT、IVF_SQ8和IVF_PQ）使用两种量化器：粗量化器与细量化器。粗量化器采用 K-均值算法（例如，在Milvus和Faiss中 K 为16384）将向量聚类为 K 个桶；细量化器则对每个桶内的向量进行编码。不同索引可能采用不同的细量化器：IVF_FLAT直接使用原始向量表示；IVF_SQ8通过采用一维量化器（即“标量量化器”）将4字节浮点数压缩为1字节整数，从而实现向量的压缩表示；而IVF_PQ则采用乘积量化，即将每个向量划分为多个子向量，并对每个子空间分别应用 K-均值算法。

基于量化的索引上的查询处理（对查询 $\mathbf{q}$）包含两个步骤：（1）根据查询 $\mathbf{q}$ 与每个桶的质心之间的距离，找出距离最近的 n_{probe} 个桶（或簇）。例如，假设图 2 中的 n_{probe} 为 2，则查询 $\mathbf{q}$ 最近的两个桶分别以 $\mathbf{c}_0$ 和 $\mathbf{c}_1$ 为中心。参数 n_{probe} 控制着精度与性能之间的权衡：n_{probe} 值越高，精度越好，但性能越差。（2）针对选出的 n_{probe} 个相关桶，分别使用不同的细量化器进行桶内搜索。例如，若图 2 中的索引为 IVF_FLAT，则需扫描这两个桶中从 $\mathbf{v}_0$ 到 $\mathbf{v}_6$ 的所有向量。

3.2 面向CPU的优化

3.2.1 Milvus 中的缓存感知优化

基于量化的索引在查询处理中面临的基本问题在于：给定一组 m 查询 $\{\mathbf{q}_1、\mathbf{q}_2、\cdots\cdots、\mathbf{q}_m\}$ 和一组 n 数据向量 $\{\mathbf{v}_1、\mathbf{v}_2、\cdots\cdots、\mathbf{v}_n\}$，如何快速

find for each query $\mathbf{q}_i$ its top-k similar vectors? In practice, users can submit batch queries so that $m \geq 1$.

This operation happens in finding the relevant buckets as well as searching within each relevant bucket. The original implementation in Facebook Faiss [3], which Milvus is built on top of, is inefficient because it incurs many CPU cache misses as explained below. Thus, Milvus develops an optimized approach to significantly reduce data movement between main memory and CPU caches.

Original implementation in Facebook Faiss [3]. Faiss uses the OpenMP multi-threading to process queries in parallel. Each thread is assigned to work on a single query at a time. The thread is released (for next query) once the current task is finished. Each task compares $\mathbf{q}_i$ with all the n data vectors and maintains a k-sized heap to store the results.

The above solution in Faiss has two performance issues: (1) It incurs many CPU cache misses, because for each query the entire data needs to be streamed through CPU caches and cannot be reused for the next query. Thus, each thread accesses m/t times of the entire data where t is the total number of threads. (2) It cannot fully leverage multi-core parallelism when the batch size m is small.

Optimizations in Milvus. Milvus develops two ideas to tackle the issues. First, it reuses the accessed data vectors as much as possible for multiple queries to minimize CPU cache misses. Specifically, it optimizes for reducing L3 cache misses because the penalty to access memory is high and also L3 cache size (typically 10s MB) is much bigger than L1/L2 cache, leaving more room for optimizations. Second, it uses fine-grained parallelism that assigns threads to data vectors instead of query vectors to best leverage multi-core parallelism, because the data size n is usually much bigger than the query size m in practice.

Figure 3 shows the overall design. Specifically, let t be the number of threads, then each thread T_i is assigned $b = n/t$ data vectors:[4] $\{\mathbf{v}_{(i-1)*b}, \mathbf{v}_{(i-1)*b+1}, ..., \mathbf{v}_{i*b-1}\}$. Milvus then partitions the m queries into query blocks of size s such that each query block (together with its associated heaps) can always fit in the L3 CPU cache. We decide s later on in Equation (1). Here we assume that m is divisible by s. Milvus computes the top-k results of each query block at a time with multiple threads. Whenever each thread loads its assigned data vectors to L3 cache, they will be compared against the entire query block (with s queries) in the cache. To minimize the synchronization overhead, Milvus assigns a heap per query per thread. In particular, assuming the i-th query block $\{\mathbf{q}_{(i-1)*s}, \mathbf{q}_{(i-1)*s+1}, ..., \mathbf{q}_{i*s-1}\}$ is in cache, Milvus dedicates the heap $H_{r-1,j-1}$ for the j-th query $\mathbf{q}_{(i-1)*s+j-1}$ on the r-th thread T_{r-1}. Thus, the results of a query $\mathbf{q}_i$ are spread over t threads of heaps. Thus, it needs to merge the heaps of each thread to obtain the final top-k results.

Next, we discuss how to determine the query block size s such that s queries and their associated heaps can always fit in L3 cache. Let d be the dimensionality, then the size of each query is $d \times$ sizeof(float). Since each heap entry contains a pair of vector ID and similarity, then the total size of the heaps (per query) is $t \times k \times$ (sizeof(int64) + sizeof(float)) where t is the number of threads. Thus, s is computed as follows:

$$s = \frac{\text{L3's cache size}}{d \times \text{sizeof(float)} + t \times k \times (\text{sizeof(int64)} + \text{sizeof(float)})}. \quad (1)$$

[4]We assume that n is divisible by t.

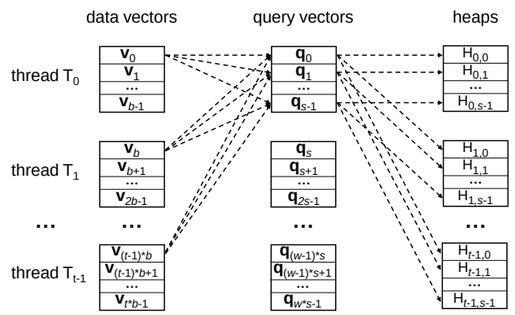


Figure 3: Cache-aware design in Milvus

In this way, each thread only accesses $m/(s*t)$ times of the entire data, which is s times smaller than the original implementation in Facebook Faiss [3]. Experiments (Sec. 7.4) show that this improves performance by a factor of 1.5× to 2.7×.

3.2.2 SIMD-aware Optimizations in Milvus

Modern CPUs support increasingly wider SIMD instructions. Thus, it is not surprising that Facebook Faiss [3] implements SIMD-aware algorithms to accelerate vector similarity search. We make two engineering optimizations in Milvus: (1) Supporting AVX512; and (2) Automatic SIMD-instruction selection.

Supporting AVX512. Faiss [3] does not support AVX512, which is now available in mainstream CPUs. Thus, we extend the similarity computing function with AVX512 instructions, such as `_mm512_add_ps`, `_mm512_mul_ps`, and `_mm512_extractf32x8_ps`. Now Milvus supports SIMD SSE, AVX, AVX2, and AVX512.

Automatic SIMD-instruction selection. Milvus is designed to work well on a wide spectrum of CPU processors (both on-premises and cloud platforms) with different SIMD instructions (e.g., SIMD SSE, AVX, AVX2, and AVX512). Thus the challenge is, given a single piece of software binary (i.e., Milvus), how to make it automatically invoke the suitable SIMD instructions on any CPU processor? Faiss [3] does not support it and users need to manually specify the SIMD flag (e.g., "-msse4") during compilation time. In Milvus, we take a considerable amount of engineering effort to refactor the codebase of Faiss. We factor out the common functions (e.g., similarity computing) that rely on SIMD accelerations. Then for each function, we implement four versions (i.e., SSE, AVX, AVX2, AVX512) and put each one into a separated source file, which is further compiled individually with the corresponding SIMD flag. During runtime, Milvus can automatically choose the suitable SIMD instructions based on the current CPU flags and then link the right function pointers using hooking.

3.3 GPU-oriented Optimizations

GPU is known for vast parallelism and Faiss [3] supports GPU for query processing over vector data. Milvus enhances Faiss in two aspects: (1) Supporting bigger k in the GPU kernel; (2) Supporting multi-GPU devices.

Supporting bigger k in GPU kernel. The original implementation in Faiss [3] does not support top-k query processing where k is greater than 1024 due to the limit of shared memory. But many

为每个查询 $\mathbf{q}_i$ 找出与其最相似的 k 个向量？在实际应用中，用户可提交批量查询，从而实现 $m \geq 1$。

该操作既发生在查找相关桶的过程中，也发生在对每个相关桶内部的搜索过程中。Milvus 所基于的 Facebook Faiss [3]，的原始实现效率低下，原因在于其会引发大量 CPU 缓存未命中，具体解释如下。因此，Milvus 开发了一种优化方法，以显著减少主存与 CPU 缓存之间的数据移动。

Facebook Faiss 中的原始实现 [3]。 Faiss 使用 OpenMP 多线程并行处理查询。每个线程一次仅负责一个查询任务；当前任务完成后，该线程即被释放（用于处理下一个查询）。每个任务将$\mathbf{q}_i$与全部 n 数据向量进行比较，并维护一个大小为 k 的堆来存储结果。

Faiss 中上述方案存在两个性能问题：（1）它会引发大量 CPU 缓存未命中，因为对于每个查询，整个数据集均需流经 CPU 缓存，且无法在后续查询中复用。因此，每个线程需访问 m/t 倍的完整数据集，其中 t 为线程总数。（2）当批处理大小 m 较小时，无法充分利用多核并行能力。

Milvus 中的优化策略。 Milvus 提出了两种思路来应对上述问题：第一，它尽可能复用已加载的数据向量，以服务多个查询，从而最大限度地减少 CPU 缓存未命中；具体而言，其优化重点在于降低 L3 缓存未命中，因为访问内存的开销较高，且 L3 缓存容量（通常为数十 MB）远大于 L1/L2 缓存，因而具备更大的优化空间。第二，它采用细粒度并行机制，将线程分配给数据向量而非查询向量，以最大程度地发挥多核并行能力，因为在实际应用中，数据大小 n 通常远大于查询大小 m。

图 3 展示了整体设计。具体而言，设 t 为线程数量，则每个线程 T_i 被分配 $b = n/t$ 个数据向量：[4] $\{\mathbf{v}_{(i-1)*b}, \mathbf{v}_{(i-1)*b+1}, ..., \mathbf{v}_{i*b-1}\}$。随后，Milvus 将 m 个查询划分为大小为 s 的查询块，使得每个查询块（连同其关联的堆）始终可完全容纳于 L3 CPU 缓存中。我们将在公式 (1) 中进一步确定 s。此处假设 m 可被 s 整除。Milvus 每次使用多个线程计算一个查询块的 top-k 结果。每当各线程将其所分配的数据向量加载至 L3 缓存后，这些数据向量即与缓存中的整个查询块（含 s 个查询）进行比对。为最小化同步开销，Milvus 为每个查询在每条线程上单独分配一个堆。具体而言，假设第 i 个查询块 $\{\mathbf{q}_{(i-1)*s}$、$\mathbf{q}_{(i-1)*s+1}$ … $\mathbf{q}_{i*s-1}\}$、… 已驻留于缓存中，则 Milvus 为第 j 个查询 $\mathbf{q}_{(i-1)*s+j-1}$ 在第 r 条线程 T_{r-1} 上专门分配堆 $H_{r-1\ j-1}$。因此，查询 $\mathbf{q}_i$ 的结果分散存储于 t 条线程各自的堆中，故需合并各线程的堆以获得最终的 top-k 结果。

接下来，我们讨论如何确定查询块大小 s，以确保 s 个查询及其关联的堆始终能够容纳在L3缓存中。设 d 为向量维度，则每个查询的大小为 $d \times \times$sizeof(float)。由于每个堆条目包含一个向量ID和一个相似度值，因此每个查询对应的堆总大小为 $t \times k \times \times$(sizeof(int64) + sizeof(float))，其中 t 为线程数。因此，s 按如下方式计算：

$$s = \frac{\text{L3's cache size}}{d \times \text{sizeof(float)} + t \times k \times (\text{sizeof(int64)} + \text{sizeof(float)})}. \quad (1)$$

[4]我们假设 n 可被 t整除。

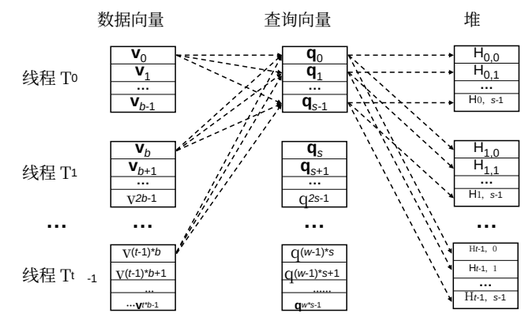


图3：Milvus中的缓存感知设计

通过这种方式，每个线程仅需访问整个数据集 $m/(s*t)$次，相比Facebook Faiss中的原始实现 [3]减少了 s 倍。实验（第7.4节）表明，该优化将性能提升了1.5× 至2.7×倍。

3.2.2 Milvus中的SIMD感知优化

现代CPU支持的SIMD指令宽度日益增加。因此，Facebook Faiss [3]采用SIMD感知算法来加速向量相似性搜索，也就不足为奇了。我们在Milvus中进行了两项工程优化：（1）支持AVX512；（2）自动选择SIMD指令。

支持AVX512。 Faiss [3] 不支持AVX512，而该指令集目前已在主流CPU中普遍可用。因此，我们通过AVX512指令（例如_mm512_add_ps、_mm512_mul_ps以及_mm512_extractf32x8_ps）扩展了相似性计算函数。目前，Milvus已支持SIMD SSE、AVX、AVX2和AVX512。

自动选择SIMD指令。 Milvus旨在广泛适配各类CPU处理器（包括本地部署及云平台环境），这些处理器所支持的SIMD指令各不相同（例如SIMD SSE、AVX、AVX2和AVX512）。因此，挑战在于：如何让单个软件二进制文件（即Milvus）在任意CPU处理器上自动调用最合适的SIMD指令？Faiss [3] 并不支持此功能，用户需在编译时手动指定SIMD标志（例如 “-msse4”）。在Milvus中，我们投入了大量工程精力对Faiss代码库进行重构：将依赖SIMD加速的通用函数（例如相似性计算）提取出来；随后，针对每个此类函数分别实现四个版本（即SSE、AVX、AVX2、AVX512），并将各版本置于独立的源文件中，再分别使用对应的SIMD标志进行单独编译。运行时，Milvus可根据当前CPU的标志位自动选择最合适的SIMD指令，并通过钩子（hooking）机制动态链接正确的函数指针。

3.3 面向GPU的优化

GPU以强大的并行处理能力著称，Faiss [3] 支持在GPU上进行向量数据的查询处理。Milvus从以下两个方面对Faiss进行了增强：（1）在GPU内核中支持更大的 k；（2）支持多GPU设备。

支持更大的 k GPU内核。 Faiss 中的原始实现 [3] 不支持 top-k 查询处理，因为共享内存限制导致k 不能超过 1024。但许多

application such as video surveillance and recommender systems may need bigger k for further verification or re-ranking [69, 71].

Milvus overcomes this limitation and supports k up to 16384 although technically Milvus can support any k.[5] When k is larger than 1024, Milvus executes the query in multiple rounds to cumulatively produce the final results. In the first round, Milvus behaves the same as Faiss and gets the top 1024 results. For the second and later rounds, Milvus first checks the distance of the last result (denoted as d_l) in the previous round. Apparently, d_l is so far the largest distance in the partial results. To handle vectors with equivalent distance to the query, Milvus also records vector IDs in the result whose distances are equal to d_l. Then Milvus filters out vectors whose distances are smaller than d_l or IDs are recorded. From the remaining data, Milvus gets the next 1024 results. By doing so, Milvus ensures that results in previous rounds will not appear in the current round. After that, the new results are merged with the partial results obtained in earlier rounds. Milvus processes the query in a round-by-round fashion until a sufficient number of results are collected.

Supporting multi-GPU devices. Faiss [3] supports multiple GPU devices since they are usually found in modern servers. But Faiss needs to declare all the GPU devices in advance during compilation time. That means if the Faiss codebase is compiled using a server with c GPUs, then the software binary can only be running in a server that has at least c GPUs.

Milvus overcomes this limitation by allowing users to select any number of GPU devices during *runtime* (instead of compilation time). As a result, once the Milvus codebase is compiled into a software binary, it can run at any server. Under the hood, Milvus introduces a segment-based scheduling that assigns segment-based search tasks to the available GPU devices. Each segment can only be served by a single GPU device. This is particularly a good fit for the cloud environment with dynamic resource management where GPU devices can be elastically added or removed. For example, if there is a new GPU device installed, Milvus can immediately discover it and assign the next available search task to it.

3.4 GPU and CPU Co-design

In this mode, the GPU memory is not large enough to store the entire data. Facebook Faiss [3] alleviates the problem by using a low-footprint compressed index (called IVF_SQ8 [3])[6] and moving data from CPU memory to GPU memory (via PCIe bus) on demand. However, we find that there are two limitations: (1) The PCIe bandwidth is not fully utilized, e.g., our experiments show that the measured I/O bandwidth is only 1~2GB/s while PCIe 3.0 (16x) supports up to 15.75GB/s. (2) It is not always beneficial to execute queries on GPU (than CPU) considering the data transfer.

Milvus develops a new index called SQ8H (where 'H' stands for hybrid) to address the above limitations (Algorithm 1).

Addressing the first limitation. We investigate the codebase of Faiss and figure out that Faiss copies data (from CPU to GPU) bucket by bucket, which underutilizes the PCIe bandwidth since each bucket can be small. So the natural idea is to copy multiple

[5]In Milvus, we purposely limit k to 16384 to prevent large data movement over networks. Also, that number is sufficient for the applications we have seen so far.

[6]Note that IVF_SQ8 takes 1/4 the space of IVF_FLAT while losing only 1% recall. But the design principle and optimizations in Sec. 3.4 can be applicable to other quantization-based indexes such as IVF_FLAT and IVF_PQ.

Algorithm 1: SQ8H

```
1 let n_q be the batch size;
2 if n_q ≥ threshold then
3     run all the queries entirely in GPU (load multiple buckets
      to GPU memory on the fly);
4 else
5     execute the step 1 of SQ8 in GPU: finding n_probe buckets;
6     execute the step 2 of SQ8 in CPU: scanning every relevant
      bucket;
```

buckets simultaneously. But the downside of such multi-bucket-copying is the handling of deletions where Faiss uses a simple in-place update approach because each bucket is copied (and stored) individually. Fortunately, deletions (and updates) are easily handled in Milvus since Milvus adopts an efficient LSM-based out-of-place approach (Sec. 2.3). As a result, Milvus improves the I/O utilization by copying multiple buckets if possible (line 3 of Algorithm 1).

Addressing the second limitation. We observe that GPU outperforms CPU only if the query batch size is large enough considering the expensive data movement. That is because more queries make the workload more computation-intensive since they search the same data. Thus, if the batch size is bigger than a threshold (e.g., 1000), Milvus executes all the queries in GPU and loads necessary buckets if GPU memory is insufficient (line 2 of Algorithm 1). Otherwise, Milvus executes the query in a hybrid manner as follows. As mentioned in Sec. 3.1, there are two steps for searching quantization-based indexes: finding n_{probe} relevant (closest) buckets and scanning each relevant bucket. Milvus executes step 1 in GPU and step 2 in CPU because we observe that step 1 has a much higher computation-to-I/O ratio than step 2 (line 5 and 6 in Algorithm 1). That is because in step 1, all the queries compare against the *same* K centroids to find n_{probe} nearest buckets, and also the K centroids are small enough to be resident in the GPU memory. By contrast, data accesses in step 2 are more scattered since different queries do not necessarily access the same buckets.

4 ADVANCED QUERY PROCESSING

4.1 Attribute Filtering

As mentioned in Sec. 2.1, attribute filtering is a hybrid query type that involves both vector data and non-vector data [65]. It only searches vectors that satisfy the attributes constraints. It is crucial to many applications [65], e.g., finding similar houses (vector data) whose sizes are within a specific range (non-vector data). For presentation purpose, we assume that each entity is associated with a single vector and a single attribute since it is straightforward to extend the algorithms to multiple attributes. We defer multi-vector query processing to Sec. 4.2.

Formally, each such query involves two conditions C_A and C_V where C_A specifies the attribute constraint and C_V is the normal vector query constraint that returns top-k similar vectors. Without loss of generality, C_A is represented in the form of $a >= p_1$ && $a <= p_2$ where a is the attribute (e.g., size, price) and p_1 and p_2 are two boundaries of a range condition (e.g., $p_1 = 100$ and $p_2 = 500$).

There are several approaches to solve attribute filtering as recently studied in AnalyticDB-V [65]. In Milvus, we implement those

视频监控和推荐系统等应用可能需要更大的 k，以进行进一步验证或重排序 [69, 71]。

Milvus 克服了这一限制，支持 k 最多达 16384；尽管从技术上讲，Milvus 可支持任意 k[5]。当 k 大于 1024 时，Milvus 将以多轮方式执行 查询，逐步累积生成最终结果。在第一轮中，Milvus 的行为与 Faiss 相同，返回前 1024 个结果。在第二轮及后续各轮中，Milvus 首先检查上一轮结果中最后一个结果（记为 d_l）的距离。显然，d_l 是当前部分结果中迄今最大的距离。为处理与 查询 距离相等的向量，Milvus 还会记录距离等于 d_l 的结果中的向量 ID。随后，Milvus 将过滤掉距离小于 d_l 或其 ID 已被记录的向量。在剩余数据中，Milvus 获取下一批 1024 个结果。通过这种方式，Milvus 确保此前各轮的结果不会在当前轮次中重复出现。之后，新获取的结果将与此前各轮获得的部分结果进行合并。Milvus 以逐轮方式处理 查询，直至收集到足够数量的结果为止。

支持多GPU设备。Faiss [3] 自现代服务器普遍配备多GPU起，即支持多个GPU设备。但Faiss需在编译时预先声明所有GPU设备。这意味着，若Faiss代码库是在配备 c 个GPU的服务器上编译的，则生成的软件二进制文件仅能在至少配备 c 个GPU的服务器上运行。

Milvus通过允许用户在运行时（而非编译时）选择任意数量的GPU设备，克服了这一限制。因此，Milvus代码库一旦编译为软件二进制文件，即可在任何服务器上运行。其底层机制引入了基于段的调度策略，将基于段的搜索任务分配给可用的GPU设备。每个段仅由单个GPU设备提供服务。这种设计特别适用于具备动态资源管理能力的云环境，其中GPU设备可弹性增减。例如，当新安装一个GPU设备时，Milvus可立即发现该设备，并将下一个待处理的搜索任务分配给它。

3.4 GPU与CPU协同设计

在此模式下，GPU内存不足以存储全部数据。Facebook Faiss [3] 通过采用低内存占用的压缩索引（称为IVF_SQ8 [3]）[6]，并按需经PCIe总线将数据从CPU内存迁移至GPU内存，缓解了该问题。然而，我们发现存在两个局限性：（1）PCIe带宽未被充分利用，例如我们的实验表明实测I/O带宽仅为 1~2GB/s，而PCIe 3.0 (16x) 的理论带宽高达15.75GB/s；（2）考虑到数据传输开销，将查询任务交由GPU（而非CPU）执行并不总是更优选择。

Milvus开发了一种新型索引SQ8H（其中"H"代表混合式），以应对上述局限性（算法1）。

解决第一个局限性。我们调研了Faiss的代码库，发现Faiss以桶为单位（从CPU向GPU）复制数据，由于每个桶可能较小，导致PCIe带宽未被充分利用。因此，一个自然的思路是同时复制多个

[5]在Milvus中，我们有意将 k 限制为16384，以避免网络上的大规模数据迁移。此外，该数值对目前所见的应用场景已足够。[6]需要注意的是，IVF_SQ8的存储空间仅为IVF_FLAT的1/4，但召回率仅下降1%。不过，第3.4节中阐述的设计原则与优化方法同样适用于其他基于量化的索引，例如IVF_FLAT和IVF_PQ。

Algorithm 1: SQ8H

```
1 let n_q be the batch size;
2 if n_q ≥ threshold then
3     run all the queries entirely in GPU (load multiple buckets
      to GPU memory on the fly);
4 else
5     execute the step 1 of SQ8 in GPU: finding n_probe buckets;
6     execute the step 2 of SQ8 in CPU: scanning every relevant
      bucket;
```


桶。但此类多桶复制方式的缺点在于删除操作的处理：Faiss采用简单的原地更新方式，因为每个桶均被单独复制（并存储）。幸运的是，Milvus中删除（及更新）操作可轻松应对，这是因为Milvus采用了高效的基于LSM的异地更新方法（第2.3节）。因此，Milvus通过在可能的情况下同时复制多个桶，提升了I/O利用率（算法1第3行）。

解决第二个局限性。我们观察到，仅当查询批大小足够大时，GPU 的性能才优于 CPU，这是因为数据传输开销高昂。其原因在于：更多的查询会使工作负载更偏向计算密集型，因为它们均需搜索相同的数据。因此，若批大小超过某一阈值（例如 1000），Milvus 将全部查询在 GPU 上执行；若 GPU 内存不足，则加载必要的桶（算法1 第2行）。否则，Milvus 采用混合方式执行查询，具体如下：如第3.1节所述，基于量化索引的搜索包含两个步骤：查找n_{probe} 相关（即最近的）桶，以及扫描每个相关桶。Milvus 在 GPU 上执行第1步，在 CPU 上执行第2步，因为我们观察到第1步的计算/输入输出比远高于第2步（算法1 第5和第6行）。这是因为在第1步中，所有查询均需与相同的 K 质心进行比较以找出n_{probe} 最近的桶，且 K 质心体积足够小，可常驻于 GPU 内存中；相比之下，第2步中的数据访问更为分散，因为不同查询未必访问相同的桶。

4 高级查询处理

4.1 属性过滤

如第 2.1 节所述，属性过滤是一种混合查询类型，同时涉及向量数据和非向量数据 [65]。它仅检索满足属性约束的向量。该技术对许多应用至关重要 [65]，，例如：查找相似房屋（向量数据），且其面积处于特定范围内（非向量数据）。为便于说明，我们假设每个实体关联一个向量和一个属性；由于将算法扩展至多个属性十分直接，因此我们暂不考虑多属性情形。多向量查询处理将在第 4.2 节中讨论。

形式化地，每个此类查询包含两个条件 C_A 和 C_V，其中 C_A 指定属性约束，C_V 是常规向量查询约束，用于返回最相似的前k 个向量。不失一般性，C_A 表示为 $a >= p_1$ && $a <= p_2$ 的形式，其中 a 为属性（例如：面积、价格），p_1 和 p_2 是范围条件的两个边界（例如：$p_1 = 100$ 和 $p_2 = 500$）。

近期在 AnalyticDB-V [65] 中对属性过滤问题的求解方法进行了多项研究，存在若干种方法。在 Milvus 中，我们实现了这些

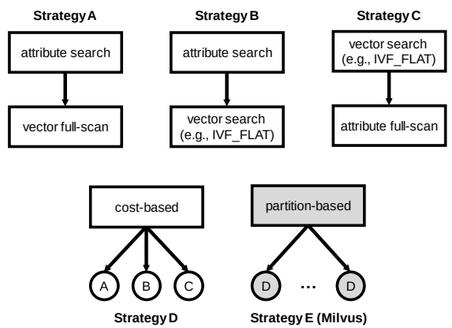


Figure 4: Different strategies for attribute filtering

approaches (i.e., strategies A, B, C, D as explained below). We then propose a partition-based approach (i.e., strategy E), which is up to 13.7× faster than the strategy D (i.e., state-of-the-art solution) according to the experiments in Sec. 7.5. Figure 4 shows a summary and we present the details next.

Strategy A: attribute-first-vector-full-scan. It only uses the attribute constraint C_A to obtain relevant entities via index search. Since the data is stored mostly in memory, we use binary search, but a B-tree index is also possible. When data cannot fit in memory, we use skip pointers for fast search. After that, all the entities in the result set are fully scanned to compare against the query vector to produce the final top-k results. Although simple, this approach is suitable when C_A is highly selective such that only a small number of candidates are required for further verification. Another interesting property of this strategy is that it produces the exact results.

Strategy B: attribute-first-vector-search. The difference with the strategy A is that after it obtains the relevant entities according to attribute constraint C_A, it produces a bitmap of the resultant entity IDs. Then it conducts the normal vector query processing based on C_V and checks the bitmap whenever a vector is encountered. Only vectors that pass bitmap testing are included in the final top-k results. This strategy is suitable in many cases when C_A or C_V is moderately selective.

Strategy C: vector-first-attribute-full-scan. In contrast to the strategy A, this approach only uses the vector constraint C_V to obtain the relevant entities via vector indexing like IVF_FLAT. Then the resultant entities are fully scanned to verify if they satisfy the attribute constraint C_A. To make sure there are k final results, it searches for $\theta \cdot k$ ($\theta > 1$) results during the vector query processing. This strategy is suitable when the vector constraint C_V is highly selective that the number of candidates is relatively small.

Strategy D: cost-based. It is a cost-based approach that estimates the cost of the strategy A, B, C, and picks up the one with the least cost as proposed in AnalyticDB-V [65]. From [65] and our experiments, the cost-based strategy is suitable in almost all cases.

Strategy E: partition-based. This is a partition-based approach that we develop in Milvus. The main idea is that it partitions the dataset based on the frequently searched attribute and applies the cost-based approach (i.e., the strategy D) for each partition. In particular, we maintain the frequency of each searched attribute in a hash table and increase the counter whenever a query refers to that attribute. Given a query of attribute filtering, it only searches the partitions whose attribute-ranges overlap with the query range. More importantly, if the range of a specific partition is covered by the query range, then this strategy does not need to check the attribute constraint (C_A) anymore and only focuses on vector query processing (C_V) in that partition, because all the vectors in that partition satisfy the attribute constraint.

As an example, suppose that there are many queries involving the attribute 'price' and the strategy E splits the dataset into five partitions: $\mathcal{P}_0$[1~100], $\mathcal{P}_1$[101~200], $\mathcal{P}_2$[201~300], $\mathcal{P}_3$[301~400], $\mathcal{P}_4$[401~500]. Then if the attribute constraint (C_A) of the query is [50~250], then only $\mathcal{P}_0$, $\mathcal{P}_1$, and $\mathcal{P}_2$ are necessary for searching because their ranges overlap with the query range. And when searching $\mathcal{P}_1$, there is no need to check the attribute constraint since its range is completely covered by the query range. This can significantly improve the query performance.

In the current version of Milvus, we create the partitions offline based on historical data and serve query processing online. The number of partitions (denoted as ρ) is a parameter configured by users. Choosing a proper ρ is subtle: If ρ is too small, then each partition contains too many vectors and it becomes hard to prune irrelevant partitions for this strategy; If ρ is too big, then the number of vectors in each partition is so small that the vector indexing deteriorates towards linear search. Based on our experience, we recommend ρ to be chosen such that each partition contains roughly 1 million vectors. For example, on a billion-scale dataset, there are around 1000 partitions. However, it is an interesting future work to investigate the use of machine learning and statistics to dynamically partition the data and decide the right number of partitions.

4.2 Multi-vector Queries

In many applications, each entity is specified by multiple vectors for accuracy. For example, intelligent video surveillance applications use different vectors to describe the front face, side face, and posture for each person captured by camera [10]. Recipe search applications use multiple vectors to represent text description and associated images for each recipe [56]. Another source of multi-vector is that many applications use more than one machine learning model even for the same object to best describe that object [30, 69].

Formally, each entity contains μ vectors $\mathbf{v}_0, \mathbf{v}_1, ..., \mathbf{v}_{\mu-1}$. Then a multi-vector query finds top-k entities according to an aggregated scoring function g over the similarity function f (e.g., inner product) of each individual vector $\mathbf{v}_i$. Specifically, the similarity of two entities X and Y is computed as $g(f(X.\mathbf{v}_0, Y.\mathbf{v}_0), ..., f(X.\mathbf{v}_{\mu-1}, Y.\mathbf{v}_{\mu-1}))$ where $X.\mathbf{v}_i$ means the vector $\mathbf{v}_i$ of the entity X. To capture a wide range of applications, we assume the aggregation function g to be monotonic in the sense that g is non-decreasing with respect to every $f(X.\mathbf{v}_i, Y.\mathbf{v}_i)$ [19]. In practice, many commonly used aggregation functions are monotonic, e.g., weighted sum, average/median, and min/max.

Naive solution. Let $\mathcal{D}$ be the dataset and $\mathcal{D}_i$ is a collection of $\mathbf{v}_i$ of all the entities, i.e., $\mathcal{D}_i = \{e.\mathbf{v}_i | e \in \mathcal{D}\}$. Given a query q, the naive solution is to issue an individual top-k query for each vector $q.\mathbf{v}_i$ on $\mathcal{D}_i$ to produce a set of candidates, which are further computed to obtain the final top-k results. Although simple, it can miss many true results leading to extremely low recall (e.g., 0.1). This approach

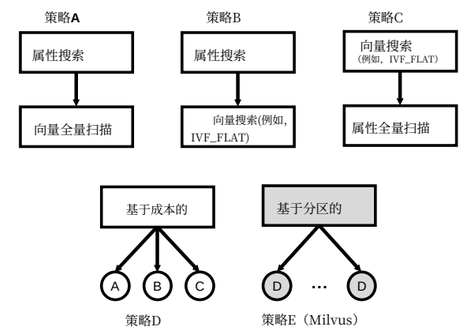


图4：不同的属性过滤策略

方法（即下文所述的策略 A、B、C、D）。随后，我们提出了一种基于分区的方法（即策略 E），根据第 7.5 节中的实验结果，其性能最高可达策略 D（即最先进方案）的 13.7× 倍。图 4 对此进行了概览，后续将详细介绍相关细节。

策略A：属性优先向量全扫描。该策略仅利用属性约束C_A，通过索引查找获取相关实体。由于数据主要存储在内存中，我们采用二分查找，但也可使用B树索引。当数据无法全部装入内存时，则使用跳指针实现快速查找。此后，对结果集中的所有实体执行全量扫描，将其与查询向量进行比对，从而生成最终的前k 个结果。尽管该方法较为简单，但在属性约束C_A具有高度选择性、仅需少量候选集即可完成后续验证的情况下尤为适用。该策略另一项有趣特性在于其可产生精确结果。

策略B：属性优先向量搜索。该策略与策略A的区别在于：在依据属性约束C_A获取相关实体后，它会生成一个结果实体ID的位图；随后基于C_V执行常规的向量查询处理，并在每次遇到一个向量时检查该位图；仅当向量通过位图检验时，才将其纳入最终的前k 个结果中。当属性约束C_A 或C_V 具有中等选择性时，该策略在许多场景下均适用。

策略C：向量优先属性全扫描。与策略A不同，该方法仅利用向量约束C_V，通过向量索引（例如IVF_FLAT）获取相关实体；随后对所得实体进行全量扫描，以验证其是否满足属性约束C_A。为确保获得 k 个最终结果，该策略在向量查询处理阶段会检索 $\theta \cdot k$ ($\theta > 1$) 个结果。当向量约束C_V 具有高度选择性、且候选集规模相对较小时，该策略尤为适用。

策略D：基于代价的。这是一种基于代价的方法，它估算策略A、B和C各自的执行代价，并选取代价最低者作为最优策略，该方案已在AnalyticDB-V [65]中提出。根据 [65] 及我们的实验结果，基于代价的策略几乎适用于所有场景。

策略E：基于分区的。这是一种我们在 Milvus 中开发的基于分区的方法。其核心思想是：根据高频查询的属性对数据集进行分区，并在每个分区内应用基于代价的方法（即策略D）。具体而言，我们使用一个哈希表来维护各被查询属性的出现频率，并在每次查询涉及某一属性时递增其计数器。

对于属性过滤类查询，该方法仅搜索其属性范围与查询范围存在重叠的分区。更重要的是，若某一分区的属性范围完全被查询范围所覆盖，则该策略无需再检查该分区内的属性约束（C_A），而只需专注于该分区内的向量查询处理（C_V），因为该分区内的所有向量均满足该属性约束。

例如，假设有大量查询涉及属性“价格”，而策略E将数据集划分为五个分区：P_0[1~100]、P_1[101~200]、P_2[201~300]、P_3[301~400]、P_4[401~500]。若查询的属性约束（C_A）为[50~250]，则仅需搜索P_0、P_1和P_2，因为这些分区的范围与查询范围存在重叠。而在搜索P_1时，无需再检查属性约束，因其范围已完全被查询范围覆盖。这可显著提升查询性能。

在当前版本的Milvus中，我们基于历史数据离线创建分区，并在线提供查询处理服务。分区数量（记为 ρ）是由用户配置的参数。选择合适的 ρ 需谨慎权衡：若 ρ 过小，则每个分区包含的向量过多，导致该策略下难以有效剪枝无关分区；若 ρ 过大，则每个分区中的向量数量过少，致使向量索引性能退化至接近线性搜索。根据我们的实践经验，建议将 ρ 设置为使每个分区大致包含100万向量。例如，在一个十亿级数据集中，分区数量约为1000个。然而，未来一项颇具意义的研究方向是探索如何利用机器学习与统计学方法，动态划分数据并智能确定最优分区数量。

4.2 多向量查询

在许多应用中，为提升准确性，每个实体由多个向量共同表征。例如，在智能视频监控应用中，针对摄像头捕获的每个人，会采用不同的向量分别描述其正面人脸、侧面人脸和姿态 [10]；食谱搜索应用则利用多个向量分别表示每道食谱的文本描述及其关联图像 [56]。多向量的另一来源是：许多应用即使针对同一对象，也会使用多个机器学习模型，以实现对该对象最全面的刻画 [30, 69]。

形式化地，每个实体包含 μ 个向量$\mathbf{v}_0$、$\mathbf{v}_1$、……、$\mathbf{v}^{\mu-1}$。多向量查询则依据聚合评分函数 g，对各单个向量$\mathbf{v}_i$的相似度函数 f（例如内积）进行综合计算，从而检索出得分最高的前k个实体。具体而言，两个实体 X与 Y之间的相似度计算公式为 $g(f(X.\mathbf{v}_0, Y.\mathbf{v}_0), ..., f(X.\mathbf{v}_{\mu-1}, Y.\mathbf{v}_{\mu-1}))$，其中 $X.\mathbf{v}_i$表示实体 X的向量$\mathbf{v}_i$。为涵盖广泛的应用场景，我们假设聚合函数 g 具有单调性，即 g关于每一个 f（$X.\mathbf{v}_i, Y.\mathbf{v}_i$）均非递减 [19]。在实际应用中，许多常用聚合函数均满足单调性，例如加权和、平均值/中位数以及最小值/最大值。

朴素解法。设 D 为数据集，D_i 是所有实体的 $\mathbf{v}_i$ 集合，即 $\mathcal{D}_i = \{e.\mathbf{v}_i | e \in \mathcal{D}\}$。给定一个查询 q，朴素解法是对 D_i 中的每个向量 $q.\mathbf{v}_i$ 分别执行一次 top-k 查询，以生成一组候选集，再对这些候选集进一步计算，从而得到最终的 top-k 结果。尽管该方法简单，却可能遗漏大量真实结果，导致召回率极低（例如仅为 0.1）。该方法

was widely used in the area of AI and machine learning to support effective recommendations, e.g., [29, 70].

In Milvus, we develop two new approaches, namely vector fusion and interactive merging that target for different scenarios.

Vector fusion. We illustrate the vector fusion approach assuming that the similarity function is inner product and we will explain how to extend to other similarity functions afterwards. Let e be an arbitrary entity in the dataset and $\mathbf{v}_0, \mathbf{v}_1, ..., \mathbf{v}_{\mu-1}$ be the μ vectors that each entity contains, this approach stores for each entity e its μ vectors as a concatenated vector $\mathbf{v} = [e.\mathbf{v}_0, e.\mathbf{v}_1, ..., e.\mathbf{v}_{\mu-1}]$. Let q be a query entity, during query processing, this approach applies the aggregation function g to the μ vectors of q, producing an aggregated query vector. For example, if the aggregation function is weighted sum with w_i for each weight, then the aggregated query vector is: $[w_0 \times q.\mathbf{v}_0, w_1 \times q.\mathbf{v}_1, ..., w_{\mu-1} \times q.\mathbf{v}_{\mu-1}]$. Then it searches the aggregated query vector against the concatenated vectors in the dataset to obtain the final results. It is straightforward to prove the correctness of vector fusion because the similarity function of inner product is decomposable.

The vector fusion approach is simple and efficient because it only needs to invoke the vector query processing once. But it requires a decomposable similarity function such as inner product. This sounds restrictive but when the underlying data is normalized, many similarity functions such as cosine similarity and Euclidean distance can be converted to inner product equivalently.

Iterative merging. If the underlying data is not normalized *and* the similarity function is not decomposable (e.g., Euclidean distance), then the above vector fusion approach is not applicable. Then we develop another algorithm called iterative merging (see Algorithm 2) that is built on top of Fagin's well-known NRA algorithm [19], a general technique for top-k query processing.[7]

Our initial try is actually to use the NRA algorithm [19] by treating the results of each $q.\mathbf{v}_i$ on $\mathcal{D}_i$ as a stream provided by Milvus. However, we quickly find that it is inefficient because NRA frequently calls `getNext()` to obtain the next result of $q.\mathbf{v}_i$ interactively. However, existing vector indexing techniques such as quantization-based indexes and graph-based indexes do not support `getNext()` efficiently. A full search is required to get the next result. Another drawback of NRA is that, it incurs significant overhead to maintain the heap since every access in NRA needs to update the scores of the current objects in the heap.

Thus, iterative merging makes two optimizations over NRA: (1) It does not rely on `getNext()` and instead calls VectorQuery($q.\mathbf{v}_i$, $\mathcal{D}_i$, k') with adaptive k' to get the top-k' query results of $q.\mathbf{v}_i$. As a result, it does not need to invoke the vector query processing for every access as NRA does. It can also eliminate the expensive overhead of the heap maintenance as in NRA. (2) It introduces an upper bound of the maximum number of steps to access since the query results in Milvus are approximate.

Algorithm 2 shows iterative merging. The main idea is that it iteratively issues a top-k' query processing for each $q.\mathbf{v}_i$ on $\mathcal{D}_i$ and puts the results to $\mathcal{R}_i$, where $\mathcal{D}_i$ is a collection of $\mathbf{v}_i$ of all the entities in the dataset $\mathcal{D}$, i.e., $\mathcal{D}_i = \{e.\mathbf{v}_i | e \in \mathcal{D}\}$, see line 3 and 4 in Algorithm 2. Then it executes the NRA algorithm over all the $\mathcal{R}_i$. If at least k results can be fully determined (line 5), i.e., NRA can safely stop, then the algorithm can terminate since top-k results

[7]Note that the TA algorithm in [19] cannot be applied in this setting because TA requires random access that is not available here.

Algorithm 2: Iterative merging

1 $k' \leftarrow k$;
2 **while** $k' < threshold$ **do**
  // run top-k' processing for each $q.\mathbf{v}_i$ on $\mathcal{D}_i$
3 **foreach** i **do**
4 $\mathcal{R}_i \leftarrow$ VectorQuery($q.\mathbf{v}_i$, $\mathcal{D}_i$, k');
5 **if** *k results are fully determined with NRA [19] on all $\mathcal{R}_i$* **then**
6 return top-k results;
7 **else**
8 $k' \leftarrow k' \times 2$;
9 return top-k results from $\cup_i \mathcal{R}_i$;

can be produced. Otherwise, it doubles k' and iterates the process until k' reaches to a pre-defined threshold (line 2).

In contrast to the vector fusion approach, the iterative merging approach makes no assumption on the data and similarity functions, thus it can be used in a wide spectrum of scenarios. But the performance will be worse than vector fusion when the similarity function is decomposable.

Note that in the database field, there are many top-k algorithms proposed, e.g., [5, 12, 31, 42, 62]. However, those algorithms cannot be directly used to solve the multi-vector query processing, because the underlying vector indexes cannot support `getNext()` efficiently as mentioned earlier. The proposed iterative merging approach (Algorithm 2) is a generic framework so that it is possible to incorporate other top-k algorithms (e.g., [42]) by replacing line 5. But it remains an open question in terms of optimality and it is also interesting to optimize multi-vector query processing in the future.

5 SYSTEM IMPLEMENTATION

In this section, we present the implementation details of asynchronous processing, snapshot isolation, and distributed computing.

5.1 Asynchronous Processing

Milvus is designed to minimize the foreground processing via asynchronous processing to improve throughput. When Milvus receives heavy write requests, it first materializes the operations (similar to database logs) to disk and then acknowledges to users. There is a background thread that consumes the operations. As a result, users may not immediately see the inserted data. To prevent this, Milvus provides an API flush() that blocks all the incoming requests until the system finishes processing all the pending operations. Besides that, Milvus builds indexes asynchronously.

5.2 Snapshot Isolation

Milvus provides snapshot isolation to make sure reads and writes see a consistent view since Milvus supports dynamic data management. Every query only works on the snapshot when the query starts. Subsequent updates to the system will create new snapshots and do not interfere with the on-going queries.

Milvus manages dynamic data following the LSM-style. All the new data are inserted to memory first and then flushed to disk as immutable segments. Each segment has multiple versions and a new

曾被广泛应用于人工智能和机器学习领域，以支持高效推荐，例如 [29, 70]。

在 Milvus 中，我们提出了两种新方法，即向量融合和交互式合并，分别面向不同应用场景。

向量融合。我们以相似度函数为内积为例说明向量融合方法，并将在后续解释如何将其扩展至其他相似度函数。设 e 为数据集中的任意一个实体，$\mathbf{v}_0$、$\mathbf{v}_1$、⋯、$\mathbf{v}^{\mu-1}$ 为该实体所包含的 μ 个向量；该方法将每个实体 e的μ个向量存储为一个拼接向量 $\mathbf{v} = [e.\mathbf{v}_0, e.\mathbf{v}_1, ..., e.\mathbf{v}_{\mu-1}]$。设 q为一个查询实体，在查询处理过程中，该方法对 q的 μ 个向量应用聚合函数 g，生成一个聚合后的查询向量。例如，若聚合函数为加权和，且各权重分别为 w_i，则聚合后的查询向量为：$[w_0 \times q.\mathbf{v}_0, w_1 \times q.\mathbf{v}_1, ..., w_{\mu-1} \times q.\mathbf{v}_{\mu-1}]$。随后，该方法将该聚合查询向量与数据集中所有拼接向量进行搜索匹配，从而获得最终结果。由于内积这一相似度函数具有可分解性，因此向量融合方法的正确性可直接证明。

向量融合方法简单且高效，因为它只需调用一次向量查询处理。但该方法要求相似度函数具备可分解性（例如内积）。这看似限制较多，但当底层数据经过归一化处理后，许多相似度函数（如余弦相似度和欧氏距离）均可等价地转换为内积形式。

迭代合并。若底层数据未归一化 且 相似度函数不具备可分解性（例如欧氏距离），则上述向量融合方法便不适用。为此，我们提出了另一种算法——迭代合并（参见算法2），该算法构建于Fagin著名的NRA算法 [19], 这一面向top-k 查询处理的通用技术之上[7]。

我们最初的尝试实际上是直接采用NRA算法 [19]，将Milvus为每个 $q.\mathbf{v}_i$ 在D_i 上返回的结果视作一个数据流。然而，我们很快发现该方案效率低下：NRA需频繁调用getNext()方法以交互式地获取 $q.\mathbf{v}_i$ 的下一个结果；而当前主流的向量索引技术（如量化索引和图索引）均无法高效支持getNext()操作，每次调用均需执行一次完整搜索才能获得下一个结果。此外，NRA的另一缺陷在于其堆维护开销显著：NRA中的每一次访问均需更新堆中当前对象的得分。

因此，迭代合并在NRA基础上实现了两项优化：（1）它不依赖于getNext()方法，而是通过自适应的 $k_\prime$ 调用 VectorQuery($q.\mathbf{v}_i$, D_i, $k_\prime$)，以获取 $q.\mathbf{v}_i$的前$k_\prime$ 个查询结果。因此，它无需像NRA那样对每次访问都执行向量查询处理，同时还能避免NRA中开销高昂的堆维护操作。（2）由于Milvus中的查询结果为近似结果，该方法引入了最大访问步数的上界。

算法2展示了迭代合并。其核心思想是：对数据集D中所有实体所构成的$\mathbf{v}_i$集合D_i（即$\mathrm{D}_i = \{e.\mathbf{v}_i | e \in \mathrm{D}\}$，参见算法2第3行和第4行），依次迭代地为每个 $q.\mathbf{v}_i$ 发出一个top-$k_\prime$ 查询处理，并将结果存入R_i；随后，在全部R_i上执行NRA算法。若至少能完全确定 k 个结果（第5行），即NRA算法可安全终止，则该算法即可结束，从而生成top-k 结果。

[7]注意：[19] 中的TA算法在此场景下无法应用，因为TA算法要求随机访问，而本场景中并不支持该访问方式。

Algorithm 2: Iterative merging

1 $k' \leftarrow k$;
2 **while** $k' < threshold$ **do**
  // run top-k' processing for each $q.\mathbf{v}_i$ on $\mathcal{D}_i$
3 **foreach** i **do**
4 $\mathcal{R}_i \leftarrow$ VectorQuery($q.\mathbf{v}_i$, $\mathcal{D}_i$, k');
5 **if** *k results are fully determined with NRA [19] on all $\mathcal{R}_i$* **then**
6 return top-k results;
7 **else**
8 $k' \leftarrow k' \times 2$;
9 return top-k results from $\cup_i \mathcal{R}_i$;

否则，将 $k_\prime$ 加倍，并重复该过程，直至 $k_\prime$ 达到预设阈值（第2行）。

与向量融合方法不同，迭代合并方法不对数据和相似度函数作任何假设，因此可广泛适用于多种场景。但当相似度函数具有可分解性时，其性能将劣于向量融合方法。

需要注意的是，在数据库领域已提出了大量前k 算法，例如 [5, 12, 31, 42, 62]。然而，这些算法无法直接用于解决多向量查询处理问题，原因在于底层向量索引如前所述无法高效支持getNext()操作。本文提出的迭代合并方法（算法2）是一种通用框架，因此可通过替换第5行来整合其他前k 算法（例如 [42]）。但该框架在最优性方面仍是一个开放问题；未来进一步优化多向量查询处理也颇具研究价值。

5 系统实现

本节将介绍异步处理、快照隔离和分布式计算的实现细节。

5.1 异步处理

Milvus 采用异步处理机制，旨在最小化前台处理开销，从而提升吞吐量。当 Milvus 接收到大量写入请求时，会首先将操作（类似于数据库日志）持久化至磁盘，随后向用户返回确认响应。与此同时，一个后台线程负责消费并处理这些已落盘的操作。因此，用户可能无法立即看到新插入的数据。为避免此问题，Milvus 提供了 API flush()，该接口会阻塞所有新进请求，直至系统完成对所有待处理操作的处理。此外，Milvus 的索引构建也是异步进行的。

5.2 快照隔离

Milvus 提供快照隔离机制，以确保读取与写入操作均能获得一致的数据视图，这是因为 Milvus 支持动态数据管理。每个查询仅作用于其启动时刻所对应的快照；后续系统更新将生成新的快照，而不会干扰正在进行中的查询。

Milvus 按照 LSM 风格管理动态数据。所有新数据首先插入内存，随后刷新至磁盘，形成不可变段。每个段具有多个版本，每当该段中的数据或索引发生变更（例如刷新、合并或构建索引时），便会生成一个新版本。

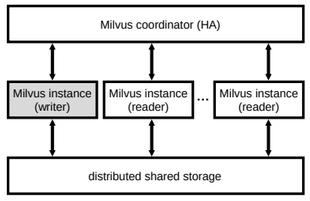


Figure 5: Milvus distributed system

version is generated whenever the data or index in that segment is changed (e.g., upon flushing, merging, or building index). All the latest segments at any time form a snapshot. Each segment can be referenced by one or more snapshots. When the system starts, there are no segments. Assuming that there are some inserts flushed to disk at t_1, which forms segment 1. Later on at t_2, segment 2 is generated. Now there are two snapshots in the system where snapshot 1 points to segment 1 and snapshot 2 points to both segment 1 and segment 2. So the segment 1 is referenced by two snapshots. All the queries before t_2 work on snapshot 1 and all the queries after t_2 work on snapshot 2. There is a background thread to garbage collect the obsolete segments if they are not referenced.

Note that the snapshot isolation is applied to the internal data reorganizations in the LSM structure. In this way, all the (internal) reads are not blocked by writes.

5.3 Distributed System

For scalability and availability, Milvus is a distributed system that supports data management across multiple nodes. From the high level, Milvus is a shared-storage distributed system that separates computing from storage to achieve the best elasticity. The shared-storage architecture is widely used in modern cloud systems such as Snowflake [16] and Aurora [63].

Figure 5 shows the overall architecture consisting of three layers. The storage layer is based on Amazon S3 (also used in Snowflake [16]) because S3 is highly available. The computing layer processes user requests such as data insertions and queries. It also has local memory and SSDs for caching data to minimize frequent accesses to S3. Besides that, there is a coordinator layer to maintain the metadata of the system such as sharding and load balancing information. The coordinator layer is highly available with three instances managed by Zookeeper.

Next, we elaborate more on the computing layer, which is stateless to achieve elasticity. It includes a single writer instance and multiple reader instances since Milvus is read-heavy and currently a single writer is sufficient to meet the customer needs. The writer instance handles data insertions, deletions, and updates. The reader instances process user queries. Data is sharded among the reader instances with consistent hashing. The sharding information is stored in the coordinator layer. There are no cross-shard transactions since there are no mixed reads and writes in the same request. The design achieves near-linear scalability as shown in the experiments (Figure 10). All the computing instances are managed by Kubernetes (K8s). When an instance is crashed, K8s will automatically restart a new instance to replace the old one. If the writer instance crashes, Milvus relies on WAL (write-ahead logging) to guarantee atomicity. Since the instances are stateless, crashing will

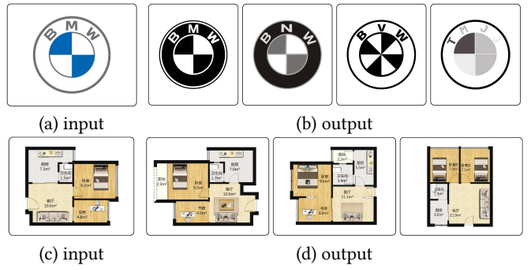


Figure 6: Milvus for image search

not affect data consistency. Besides that, K8s can also elastically add more reader instances if existing ones are overloaded.

To minimize the network overhead between computing and storage, Milvus employs two optimizations: (1) The computing layer only sends logs (rather than the actual data) to the storage layer, similar to Aurora [63]. As mentioned in Sec. 5.1, Milvus asynchronously processes the logs with a background thread to improve performance. In the current implementation, the background thread comes from the writer instance since the writer's load is not too high. Otherwise, the log processing can be managed by a dedicated instance. (2) Another optimization is that each computing instance has a significant amount of buffer memory and SSDs to reduce accesses to the shared storage.

6 APPLICATIONS

In this section, we present applications that are powered by Milvus. We have built **10 applications** on top of Milvus that includes image search, video search, chemical structure analysis, COVID-19 dataset search, personalized recommendation, biological multi-factor authentication, intelligent question answering, image-text retrieval, cross-modal pedestrian search, and recipe-food search. This section presents two of them due to space limit and more can be found in *https://github.com/milvus-io/bootcamp/tree/master/EN_solutions*.

6.1 Image Search

Image search is a well known application of vector search where each image is naturally converted to a vector using deep learning models such as VGG [58] and ResNet [28].

Two tech companies, Qichacha[8] and Beike Zhaofang,[9] currently use Milvus for large-scale image searches. Qichacha is a leading Chinese website for storing and searching business information (of over 100 million companies), e.g., the names of officers/shareholders and credit information. Milvus supports Qichacha in finding similar trademarks for customers to check if their trademarks have been registered. Beike Zhaofang is one of the biggest online real estate transaction platform in China. Milvus supports Beike Zhaofang in finding similar houses and apartments (e.g., floor plans). Figure 6 shows an example of searching business trademarks and houses in Qichacha and Beike Zhaofang using Milvus.

[8]https://www.qcc.com/
[9]https://www.ke.com/

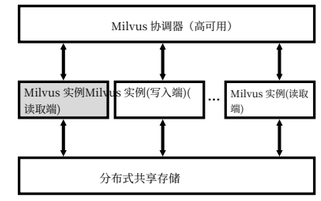


图5：Milvus 分布式系统

在任意时刻，所有最新版本的段共同构成一个快照。每个段可被一个或多个快照所引用。系统启动时不存在任何段。假设在 t_1 时刻，部分插入操作已刷新至磁盘，由此生成段 1；随后在 t_2 时刻生成段 2。此时系统中存在两个快照：快照 1 指向段 1，而快照 2 同时指向段 1 和段 2，因此段 1 被两个快照所引用。所有在 t_2 之前发起的查询均基于快照 1 执行，而所有在 t_2 之后发起的查询则基于快照 2 执行。系统中存在一个后台线程，用于对未被任何快照引用的过时段执行垃圾回收。

注意，快照隔离应用于 LSM 结构中的内部数据重组。通过这种方式，所有（内部）读操作均不会被写操作阻塞。

5.3 分布式系统

为实现可扩展性和高可用性，Milvus 是一种分布式系统，支持跨多个节点的数据管理。从高层视角看，Milvus 是一种共享存储分布式系统，通过将计算与存储分离以实现最佳弹性。共享存储架构在现代云系统（例如 Snowflake [16] 和 Aurora [63]）中被广泛应用。

图5展示了由三层组成的整体架构。存储层基于 Amazon S3（Snowflake [16] 中也使用了该服务），因为 S3 具有极高的可用性。计算层负责处理用户请求，例如数据插入和查询；同时配备本地内存和固态硬盘（SSD），用于缓存数据，从而最大限度地减少对 S3 的频繁访问。此外，还存在一个协调器层，用于维护系统的元数据（例如分片和负载均衡信息）。该协调器层通过 Zookeeper 管理三个实例，具备高可用性。

接下来，我们进一步阐述计算层。该层采用无状态设计，以实现弹性伸缩能力。由于 Milvus 以读操作为主，且当前单个写入实例已足以满足客户需求，因此计算层包含一个写入实例和多个读取实例。写入实例负责处理数据的插入、删除与更新操作；读取实例则负责响应用户的查询请求。数据通过一致性哈希在各读取实例间进行分片，分片信息存储于协调器层中。由于单个请求中不会同时包含读操作与写操作，因此不存在跨分片事务。如实验结果（图 10）所示，该设计可实现近线性扩展能力。所有计算实例均由 Kubernetes（K8s）统一管理：当某个实例发生崩溃时，K8s 将自动启动一个新实例以替代原实例；若写入实例崩溃，Milvus 则依赖预写日志（WAL）来保障操作的原子性。由于各实例均为无状态，因此崩溃事件

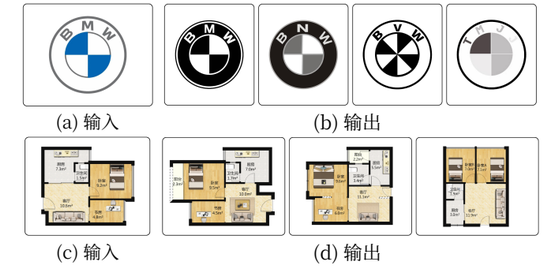


图6：Milvus 用于图像搜索

不会影响数据一致性。此外，当现有读取实例负载过高时，K8s 还可弹性地动态增加更多读取实例。

为最小化计算层与存储层之间的网络开销，Milvus 采用了两项优化措施：（1）计算层仅向存储层发送日志（而非实际数据），其设计思路类似于 Aurora [63]。如第 5.1 节所述，Milvus 通过后台线程异步处理日志以提升性能。在当前实现中，该后台线程来自写入器（writer）实例，因为写入器的负载并不高；否则，日志处理可交由专用实例来管理。（2）另一项优化是，每个计算实例均配备大量缓冲内存和固态硬盘（SSD），以减少对共享存储的访问。

6 应用场景

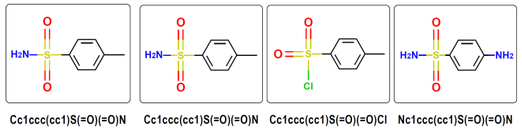
本节介绍由 Milvus 驱动的应用场景。我们已在 Milvus 基础上构建了**10 个应用**，涵盖图像搜索、视频搜索、化学结构分析、COVID-19数据集搜索、个性化推荐、生物多因子身份认证、智能问答、图文检索、跨模态行人搜索以及食谱-食物搜索。受篇幅所限，本节仅介绍其中两个应用；更多应用可参见 *https://github.com/milvus-io/bootcamp/tree/master/EN_solutions*。

6.1 图像搜索

图像搜索是向量搜索的一种广为人知的应用场景，其中每张图像均可借助 VGG [58] 和 ResNet [28] 等深度学习模型自然地转换为向量。

两家科技公司——企查查[8] 和贝壳找房[9] ——目前均使用 Milvus 进行大规模图像搜索。企查查是中国领先的商业信息存储与检索中文网站，收录了超过一亿家企业的信息，例如企业高管/股东姓名及信用信息。Milvus 为企查查提供支持，帮助客户查找相似商标，以确认其商标是否已被注册。贝壳找房是中国最大的在线房产交易平台之一。Milvus 为贝壳找房提供支持，用于查找相似的房屋与公寓（例如户型图）。图6展示了企查查和贝壳找房分别利用 Milvus 搜索商业商标与房屋的示例。

[8]https://www.qcc.com/ [9]
https://www.ke.com/



(a) input (b) output

Figure 7: Milvus for chemical structure analysis

6.2 Chemical Structure Analysis

Chemical structure analysis is an emerging application that depends on vector search. Recent studies have demonstrated that a new efficient paradigm of understanding the structure of a chemical substance is to encode it into a high-dimensional vector and use vector similarity search (e.g., with Tanimoto distance [9]) to find similar structures [9, 66].

Milvus is now adopted by Apptech,[10] a major pharmaceutical company developing new medicines and medical devices. Milvus significantly reduces the time of chemical structure analysis from hours to less than a minute. Figure 7 shows an example of searching similar chemical structures using Milvus.

7 EXPERIMENTS

7.1 Experimental Setup

Experimental platform. We conduct all the experiments on Alibaba Cloud and use different types of computing instances (up to 12 nodes) for different experiments to save monetary cost. By default, we use the CPU instance of ecs.g6e.4xlarge (Xeon Platinum 8269 Cascade 2.5GHz, 16 vCPUs, 35.75MB L3 cache, AVX512, 64GB memory, and NAS elastic storage). The GPU instance is ecs.gn6i-c16g1.4xlarge (NVIDIA Tesla T4, 64KB private memory, 512KB local memory, 16GB global memory, and PCIe 3.0 16x interface).

Datasets. To be reproducible, we use the following two public datasets to evaluate Milvus: SIFT1B [34] and Deep1B [8]. SIFT1B contains 1 billion 128-dimensional SIFT vectors (512GB) and Deep1B contains 1 billion 96-dimensional image vectors (384GB) extracted from a deep neural network. Both are standard datasets used in many previous works on vector similarity search and approximate nearest neighbor search [35, 41, 65, 68].

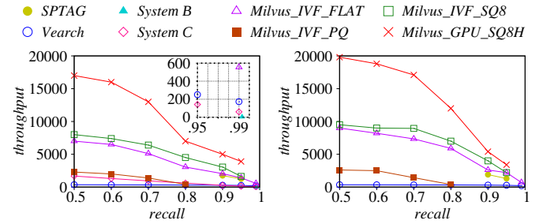
Competitors. We compare Milvus against two open-source systems: Jingdong Vearch (v3.2.0) [4, 39] and Microsoft SPTAG [14]. We also compare Milvus with three industrial-strength commercial systems (with latest version as of July 2020) anonymized as System A, B, and C for commercial reasons. Since Milvus is implemented on top of Faiss [3, 35], we also present the performance comparison by evaluating the algorithmic optimizations in Milvus (Sec. 7.4).

Evaluation metrics. We use the recall to evaluate the accuracy of the top-k results returned by a system where k is 50 by default. Specifically, let S be the ground-truth top-k result set and S' be the top-k results from a system, then the recall is defined as $|S \cap S'|/|S|$. Besides that, we also measure the throughput of a system by issuing 10,000 random queries to the datasets.

7.2 Comparing with Prior Systems

In this experiment, we compare Milvus against prior systems in terms of recall and throughput. We use the first 10 million vectors

[10] https://www.wuxiapptec.com/



(a) SIFT10M (b) Deep10M

Figure 8: System evaluation on IVF indexes

from each dataset (referred to as SIFT10M and Deep10M) because prior systems are slow in building indexes and executing queries on billion-scale datasets. Note that we also evaluate Milvus on the full billion-scale vectors in Sec. 7.3 to demonstrate the system scalability. Except for the three commercial systems (A, B, and C) that the minimum configuration requires multiple nodes, we run all other systems (including Milvus) in a single node. Specifically, we run System A and C on two nodes (with 64GB memory per node); System B on four nodes (with 128GB memory per node).

In this experiment, we use two indexes IVF_FLAT and HNSW whenever possible since both are supported by most systems, although Milvus supports more indexes.

Figure 8 shows the results on IVF indexes (i.e., quantization-based indexes). Overall, Milvus (even CPU version) significantly outperforms existing systems by up to two orders of magnitude while keeping the similar recall. In particular, Milvus is 6.4× ~ 27.0× faster than Vearch; 153.7× faster than System B even if System B runs on four nodes;[11] 4.7× ~ 11.5× faster than System C even if System C runs on two nodes; 1.3× ~ 2.1× faster than SPTAG (tree-based index). But SPTAG cannot achieve very high recall (e.g., 0.99) as Milvus does and also SPTAG takes 14× more memory than Milvus (17.88GB vs. 1.27GB).[12] The GPU version of Milvus is even faster since data can fit in the GPU memory in this setting. We omit the results of System B on Deep10M since it only supports the Euclidean distance metric. We also omit the results of Vearch on GPU because there are multiple bugs in building indexes that their engineers were still fixing by the time of paper writing.[13] We defer the results of System A to Figure 9 since it only supports the HNSW index.

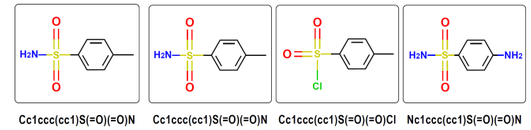
The performance advantage of Milvus comes from a few factors in addition to engineering optimizations. (1) Milvus introduces fine-grained parallelism that supports both inter-query and intra-query parallelism to best leverage multi-core CPUs. (2) Milvus develops cache-aware and SIMD-aware optimizations to reduce CPU cache misses and leverage wide SIMD instructions. (3) Milvus optimizes for the hybrid execution between GPU and CPU.

Figure 9 shows the results on the HNSW index of each system. Milvus outperforms existing systems by a large margin. Specifically, it is 15.1× ~ 60.4× faster than Vearch; 8.0× ~ 17.1× faster than

[11] Note that System B has a single data point in Figure 8 and relatively low performance because it used brute-force search as it disabled the parameter tuning (e.g., n_{probe} and n_{list}) when we tested in 08/2020. But we expect a better performance in System B once the parameter tuning is enabled (to use index) in the future.

[12] Besides that, SPTAG does not support dynamic data management, GPU, attribute filtering, multi-vector query, and distributed systems that Milvus provides, see Table 1.

[13] We submitted a bug report in 09/2020: *https://github.com/vearch/vearch/issues/252*.



（a）输入 （b）输出

图 7：Milvus 在化学结构分析中的应用

6.2 化学结构分析

化学结构分析是一种新兴应用，依赖于向量搜索技术。近期研究表明，理解化学物质结构的一种新型高效范式是：将其编码为高维向量，并利用向量相似性搜索（例如采用 Tanimoto 距离 [9]）来查找结构相似的化合物 [9, 66]。

Milvus 目前已被制药公司 Apptech[10] 采用，该公司是一家致力于研发新药及医疗器械的大型制药企业。Milvus 将化学结构分析所需时间从数小时大幅缩短至不到一分钟。图 7 展示了使用 Milvus 搜索相似化学结构的一个示例。

7 实验

7.1 实验设置

实验平台。所有实验均在阿里云上进行，针对不同实验采用不同类型（最多12个节点）的计算实例，以节省成本。默认情况下，我们使用 ecs.g6e.4xlarge CPU 实例（ Xeon Platinum 8269 Cascade 2.5GHz、16 个 vCPU、35.75MB L3缓存、AVX512、64GB 内存及 NAS 弹性存储）。GPU 实例为 ecs.gn6i-c16g1.4xlarge（ NVIDIA Tesla T4、64KB 私有内存、512KB 本地内存、16GB 全局内存以及 PCIe 3.0 16x 接口）。

数据集。为确保可复现性，我们采用以下两个公开数据集对 Milvus 进行评估：SIFT1B [34] 和 Deep1B [8]。SIFT1B 包含 10 亿个 128 维 SIFT 向量（512GB），Deep1B 包含 10 亿个 96 维图像向量（384GB），这些图像向量由深度神经网络提取。这两个数据集均为向量相似性搜索与近似最近邻搜索领域诸多前期研究中所采用的标准数据集 [35, 41, 65, 68]。

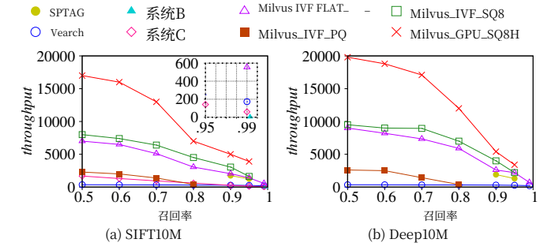
竞品对比。我们将 Milvus 与两个开源系统进行对比：京东 Vearch（v3.2.0） [4, 39] 和 Microsoft SPTAG [14]。此外，我们还将 Milvus 与三个工业级商用系统（截至 2020 年 7 月的最新版本）进行对比；出于商业原因，这些系统被匿名化为系统 A、系统 B 和系统 C。由于 Milvus 基于 Faiss [3, 35], 实现，我们还通过评估 Milvus 中的算法优化（第 7.4 节）来呈现其性能对比。

评估指标。我们采用召回率来评估系统返回的前 k 个结果的准确性，其中 k 默认为 50。具体而言，设 S 为真实情况下的前 k 个结果集合， $S\prime$ 为某系统返回的前 k 个结果，则召回率定义为 $|S \cap S\prime|/|S|$。此外，我们还通过向数据集发送 10,000 个随机查询来测量系统的吞吐量。

7.2 与先前系统的对比

在本实验中，我们从召回率和吞吐量两方面对 Milvus 与现有系统进行对比。我们使用每个数据集的前 1000 万个向量

[10] https://www.wuxiapptec.com/



(a) SIFT10M (b) Deep10M

图8：IVF索引上的系统评估

（分别称为 SIFT10M 和 Deep10M），因为现有系统在十亿级数据集上构建索引和执行查询的速度较慢。需要注意的是，我们在第 7.3 节中还使用完整的十亿级向量对 Milvus 进行了评估，以展示系统的可扩展性。除三个商业系统（系统 A、系统 B 和系统 C）外，其余所有系统（包括 Milvus）均在单节点上运行；这三个商业系统的最低配置要求为多节点部署。具体而言，系统 A 和系统 C 在两个节点上运行（每个节点配备 64 GB 内存）；系统 B 在四个节点上运行（每个节点配备 128 GB 内存）。

在本实验中，我们尽可能使用两种索引：IVF_FLAT 和 HNSW，因为大多数系统均支持这两种索引，尽管 Milvus 支持更多类型的索引。

图8展示了IVF索引（即基于量化的索引）上的实验结果。总体而言，Milvus（即使是CPU版本）在保持相近召回率的前提下，性能显著优于现有系统，最高可达两个数量级。具体而言，Milvus比Vearch快6.4× ~ 27.0× 倍；即使System B运行在四节点集群上，Milvus仍比其快153.7× 倍；即使System C运行在双节点集群上，Milvus仍比其快114.7× ~ 11.5× 倍；Milvus也比SPTAG（基于树的索引）快1.3× ~ 2.1× 倍。但SPTAG无法达到Milvus所能实现的极高召回率（例如0.99），且其内存占用比Milvus高出 14×（17.88 GB vs. 1.27 GB）。[12] Milvus的GPU版本速度更快，因为在此设定下数据可完全载入GPU内存。由于System B仅支持欧氏距离度量，我们未在Deep10M数据集上展示其结果。我们亦未展示Vearch在GPU上的结果，因其索引构建过程中存在多个Bug，截至本文撰写时，其工程师仍在修复中。[13] System A的结果延后至图9展示，因其仅支持HNSW索引。

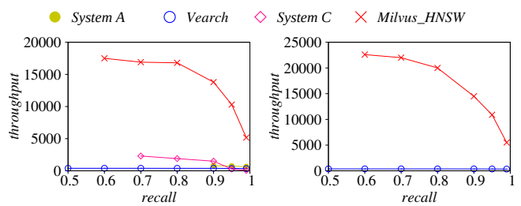
Milvus 的性能优势不仅源于工程优化，还来自以下几个方面：（1）Milvus 引入了细粒度并行机制，同时支持查询间并行与查询内并行，从而最大限度地利用多核 CPU；（2）Milvus 开发了缓存感知与 SIMD 感知优化技术，以减少 CPU 缓存未命中并充分利用宽 SIMD 指令；（3）Milvus 针对 GPU 与 CPU 的混合执行进行了专门优化。

图9展示了各系统在HNSW索引上的结果。Milvus大幅优于现有系统：具体而言，其性能比Vearch快15.1× ~ 60.4× 倍；比

[11] 注意：系统B 在图 8 中仅有一个数据点，且性能相对较低，原因在于我们在 2020 年 8 月测试时禁用了参数调优（例如 n_{probe} 和 n_{list}），因而采用了暴力搜索。但我们预计，未来启用参数调优（以使用索引）后，系统B 的性能将得到提升。

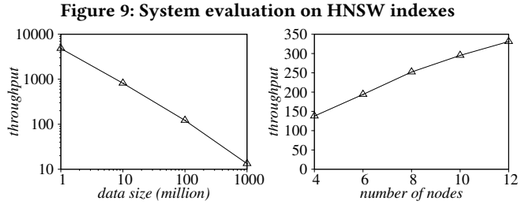
[12] 此外，SPTAG 不支持 Milvus 所提供的动态数据管理、GPU 加速、属性过滤、多向量查询及分布式系统等功能，详见表 1。

[13] 我们已于 2020 年 9 月提交一份缺陷报告：*https://github.com/vearch/vearch/issues/252*。



(a) SIFT10M (b) Deep10M

Figure 9: System evaluation on HNSW indexes



(a) Varying data size (b) Varying #nodes

Figure 10: Scalability

System A; 7.3× ~ 73.9× faster than System C. We omit System A on Deep10M because System A does not support inner product. We omit also System C on Deep10M because the index building fails to complete after more than 100 hours.

7.3 Scalability

In this experiment, we evaluate the scalability of Milvus in terms of data size and the number of servers. We use the IVF_FLAT indexing on the SIFT1B dataset that includes 1 billion vectors.

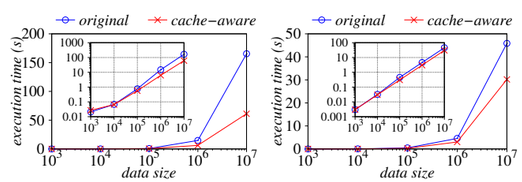
Figure 10a shows the results on a single node of ecs.re6.26xlarge (104 vCPUs and 1.5TB memory) that can fit the entire data in memory. As the data increases, the throughput gracefully drops proportionally. Figure 10b shows the scalability of distributed Milvus. The data is sharded among the nodes where each node is of ecs.g6e.13xlarge (52 vCPUs and 192GB memory). As the number of nodes increases, the throughput increases linearly. Note that we observe that Milvus achieves higher throughput on the ecs.g6e.13xlarge instance than the ecs.re6.26xlarge instance due to the higher competition on the shared CPU caches and memory bandwidth among more cores.

7.4 Evaluation of Optimizations

Figure 11 shows the impact of cache-aware design on two CPUs with different L3 cache sizes: 12MB (Intel Core i7-8700 3.2GHz) and 35.75MB (Xeon Platinum 8269 Cascade 2.5GHz). We set the query batch size as 1000 and vary the data size (i.e., the number of vectors) from 1000 to 10 million. It shows that the cache-aware design can achieve a performance improvement up to 2.7× and 1.5× when the cache size is 12MB and 35.75MB, respectively.

Figure 12 shows the impact of SIMD-aware optimizations following the experimental setup in Figure 11. It compares the performance of AVX2 and AVX512 on the Xeon CPU. Figure 12 demonstrates that AVX512 is roughly 1.5× faster than AVX2.

Figure 13 evaluates the efficiency of the hybrid algorithm SQ8H (Algorithm 1) in Milvus on SIFT1B where data cannot fit into GPU



(a) 12MB L3 cache (b) 35.75MB L3 cache

Figure 11: Evaluating the cache-aware design

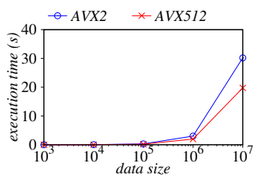


Figure 12: SIMD optimizations

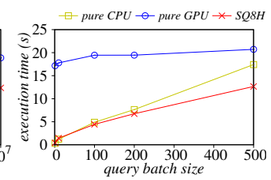


Figure 13: GPU indexing

memory. We compare SQ8H with SQ8 on pure CPU and pure GPU. It shows that GPU SQ8 is slower than CPU SQ8 due to the data transfer. As the query batch size increases, the performance gap between GPU and CPU becomes smaller since more computations are pushed to the GPU. In all cases, SQ8H is faster than running SQ8 on pure CPU and pure GPU. That is because SQ8H only stores the centroids in GPU memory to execute the first step and allows the CPU to execute the second step so that there is no any data segment transferred to GPU memory on the fly.

7.5 Evaluating Attribute Filtering

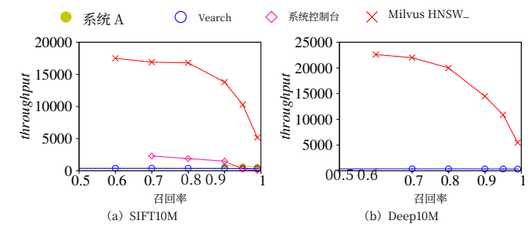
We define the query selectivity as the percentage of entities that fails the attribute constraint C_A following [65]. Thus, a higher selectivity means that a smaller number of entities can pass C_A. Regarding the dataset, we extract the first 100 million vectors from SIFT1B and augment each vector with an attribute of a random value ranging from 0 to 10000. We follow [65] to generate two scenarios of different k (50 and 500) and recall (0.95 and 0.85).

Figure 14 shows the results with varying query selectivity. For the strategy A, its performance increases as the selectivity increases because the number of examined vectors decreases. The strategy B is insensitive to the selectivity since the bottleneck is vector similarity search. The strategy C is slower than the strategy B since it requires to check θ times of the vectors where θ is 1.1 in this experiment. The strategy D outperforms A, B, and C since it uses a cost-based approach to choose the best between the three. Our new approach, i.e., the strategy E, significantly outperforms the strategy D by up to 13.7× due to the partitioning.

Figure 15 compares Milvus against System A, B, C, and Vearch in terms of attribute filtering. It shows that Milvus outperforms those systems by 48.5× to 41299.5×. Note that we omit the results of System B in Figure 15b because its parameters are fixed by the system that users are not allowed to change.

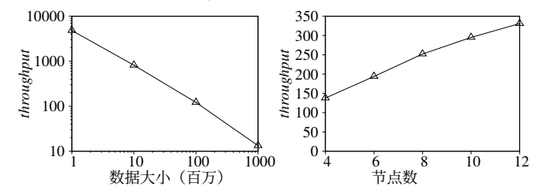
7.6 Evaluating Multi-vector Query Processing

In this experiment, we evaluate the algorithms for multi-vector query processing. Since SIFT1B and Deep1B only contain one vector



(a) SIFT10M (b) Deep10M

图9：HNSW索引上的系统评估



(a) 数据大小变化 (b) 节点数量变化

图10：可扩展性

系统 A快8..倍；比系统 C快7.3× ~ 73.9× 倍。我们在Deep10M数据集上未测试系统 A，因为系统 A不支持内积；同样，我们也在Deep10M数据集上未测试系统 C，因为其索引构建在耗时超过100小时后仍未完成。

7.3 可扩展性

在本实验中，我们从数据大小和服务器数量两个维度评估 Milvus 的可扩展性。我们在包含 10 亿个向量的 SIFT1B 数据集上使用 IVF_FLAT 索引构建方法。

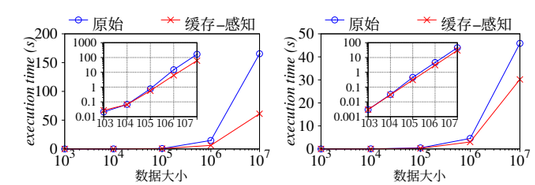
图10a展示了单节点 ecs.re6.26xlarge（104 个 vCPU、1.5 TB 内存）上的测试结果，该节点内存足以容纳全部数据。随着数据量增加，吞吐量呈比例地平缓下降。图10b展示了分布式 Milvus 的可扩展性：数据在各节点间分片存储，每个节点为 ecs.g6e.13xlarge（52 个 vCPU、192 GB 内存）。随着节点数量增加，吞吐量呈线性增长。需要注意的是，我们观察到 Milvus 在 ecs.g6e.13xlarge 实例上实现的吞吐量高于 ecs.re6.26xlarge 实例，这是因为后者拥有更多 CPU 核心，导致共享 CPU 缓存与内存带宽的竞争更为激烈。

7.4 优化效果评估

图11展示了缓存感知设计在两种L3缓存大小不同的CPU上的影响：12 MB（Intel Core i7-8700 3.2GHz）和35.75 MB（Xeon Platinum 8269 Cascade 2.5GHz）。我们将查询批大小设为1000，并将数据大小（即向量数量）从1000变化至1000万。结果表明，当缓存大小分别为12 MB和35.75 MB时，缓存感知设计所能实现的性能提升最高分别可达2.7× 倍和1.5× 倍。

图12展示了SIMD感知优化的影响，其实验设置与图11一致。该图对比了Xeon CPU上AVX2与AVX512的性能表现。图12表明，AVX512的性能约为AVX2的1.5×倍。

图13评估了混合算法SQ8H（算法1）在Milvus中处理SIFT1B数据集时的效率，该数据集无法完全载入GPU内存。



(a) 12 MB L3缓存 (b) 35.75 MB L3缓存

图11：评估缓存感知设计

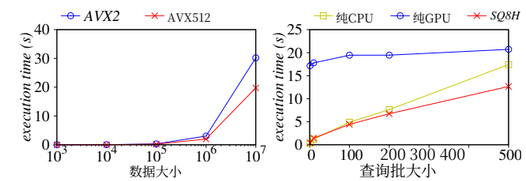


图12：SIMD优化 **图13：GPU索引构建**

我们将SQ8H与纯CPU和纯GPU上的SQ8进行了对比。结果表明，由于数据传输开销，GPU版SQ8比CPU版SQ8更慢。随着查询批大小的增加，GPU与CPU之间的性能差距逐渐缩小，因为更多的计算被卸载至GPU执行。在所有情况下，SQ8H均快于纯CPU和纯GPU上运行的SQ8。其原因在于：SQ8H仅将质心存储于GPU内存中以执行第一步，而第二步则交由CPU执行，从而避免了在运行时将任何数据段传输至GPU内存。

7.5 评估属性过滤

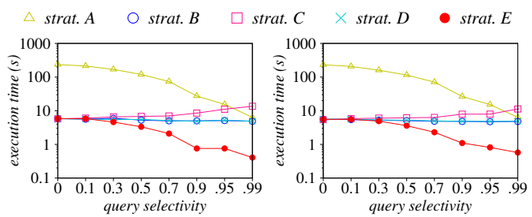
我们将查询选择性定义为无法满足属性约束 C_A 的实体所占的百分比，该约束位于 [65]之后。因此，选择性越高，表示能通过约束 C_A的实体数量越少。关于数据集，我们从 SIFT1B 中提取前 1 亿个向量，并为每个向量附加一个取值范围为 0 至 10000 的随机属性值。我们依据 [65] 生成两种不同查询选择性（分别为 50 和 500）和召回率（分别为 0.95 和 0.85）的场景。

图 14 展示了在不同查询选择性下的实验结果。对于策略 A，其性能随选择性的升高而提升，这是因为被检查的向量数量随之减少。策略 B 对选择性不敏感，因为其性能瓶颈在于向量相似性搜索。策略 C 比策略 B 更慢，因为它需要检查 θ 倍的向量，而本实验中 θ 的值为 1.1。策略 D 优于策略 A、B 和 C，因为它采用基于代价的方法，在三者之中选择最优方案。我们提出的新方法，即策略 E，得益于分区机制，其性能显著优于策略 D，最高可提升达 13.7× 。

图15从属性过滤角度对比了Milvus与系统A、系统B、系统C及Vearch的性能。结果表明，Milvus的性能优于这些系统，提升幅度为48.5× 至41299.5×。请注意，由于系统B的参数由系统固定且用户无法修改，因此我们在图15b中省略了系统B的结果。

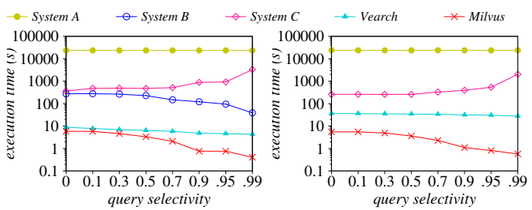
7.6 多向量查询处理评估

在本实验中，我们评估了多向量查询处理算法。由于 SIFT1B 和 Deep1B 数据集仅包含一个向量，



(a) k = 50 & recall >= 0.95 (b) k = 500 & recall >= 0.85

Figure 14: Attribute filtering in Milvus



(a) k = 50 & recall >= 0.95 (b) k = 500 & recall >= 0.85

Figure 15: Attribute filtering comparison

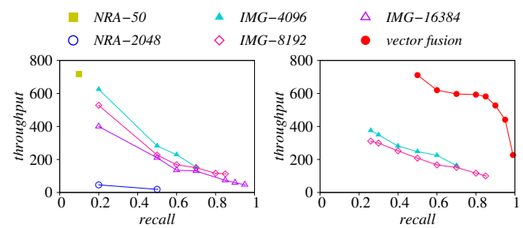
per entity, we then use another dataset called Recipe1M [50, 56] that includes more than one million cooking recipes and food images. Thus, each entity is described by two vectors: text vector (i.e., recipe description) and image vector (i.e., food image). We randomly pick up 10000 queries from the dataset and set k as 50 in this experiment. We use the IVF_FLAT indexing in this experiment. Besides that, we use weighted sum as the aggregation function.

Figure 16a shows the results where the similarity metric is Euclidean distance. We compare the standard NRA algorithm of different k (50 and 2048) and our iterative merging ("IMG" for short) of different k' (4096, 8192, and 16384). It shows that the standard NRA approach is either slow or produces low recall. In particular, the NRA-50 approach is fast but the recall is only 0.1. The NRA-2048 increases the recall a bit (up to 0.5), but the performance is low while our iterative merging algorithm (with k' being 4096) is 15× faster than NRA-2048 with a similar recall. That is because IMG does not need to invoke the vector query processing every time and also it has lower maintenance cost of heaps.

Figure 16b shows the results on the inner product metric. We compare the iterative merging (IMG-4096 and IMG-8192) with vector fusion. It shows that vector fusion is 3.4× ~ 5.8× faster since it only needs to issue a single top-k vector similarity search.

8 RELATED WORK

Vector similarity search (a.k.a high-dimensional nearest neighbor search) is an extensively studied topic both for approximate search (e.g., [7, 41]) and exact search (e.g., [38, 42]). This work focuses on approximate search in order to achieve high performance. Prior works on approximate search can be roughly classified in four categories: LSH-based [23, 23, 24, 32, 40, 44, 45, 48, 73], tree-based [17, 46, 54, 57], graph-based [20, 43, 49, 61, 72], and quantization-based [3, 6, 22, 27, 33, 35]. However, those works are



(a) Euclidean distance (b) Inner product

Figure 16: Multi-vector processing in Milvus

all about indexes while Milvus is a full-fledged vector data management system including indexes, query engine, GPU engine, storage engine, and distributed system. Moreover, Milvus's extensible index framework can easily incorporate those indexes as well as any new index if necessary. There are also open-source libraries for vector similarity search, e.g., Faiss [35] and SPTAG [14]. But they are libraries not systems. We summarize the differences in Table 1.

Recent industrial-strength vector data management systems such as Alibaba PASE [68] and Alibaba AnalyticDB-V [65] are not particularly optimized for vectors. Their approach is to extend the relational database to support vectors. As a result, the performance suffers severely as demonstrated in experiments. Specialized vector systems like Vearch [39] are not suitable for billion-scale data and Vearch is significantly slower than Milvus.

There are also GPU-based vector search engines, e.g., [35, 72]. Of which, [72] optimizes HNSW for GPU but it assumes data to be small enough to fit into GPU memory. Faiss [35] also supports GPU, but it loads the whole data segments on demand if data cannot fit into GPU memory, leading to low performance. Instead, Milvus develops a new hybrid index (SQ8H) that combines the best of the GPU and CPU without loading data on the fly for fast query processing.

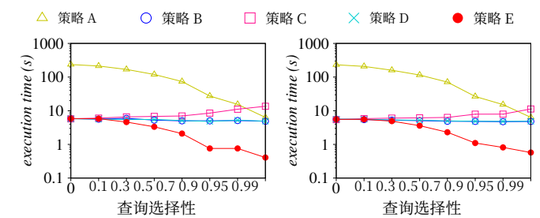
This work is relevant to the trend of building specialized data engines since one size does not fit all [60], e.g., specialized graph engine [18], IoT engine [21], time series database [55], and scientific database [59]. In this regard, Milvus is a specialized data engine for managing vector data.

9 CONCLUSION

In this work, we share our experience in building Milvus over the last few years at Zilliz. Milvus has been adopted by hundreds of companies and is currently an incubation-stage project at the LF AI & Data Foundation. Looking forward, we plan to leverage FPGA to accelerate Milvus. We have implemented the IVF_PQ indexing on FPGA and the initial results are encouraging. Another interesting yet challenging direction is to architect Milvus as a cloud-native data management system and we are currently working on it.

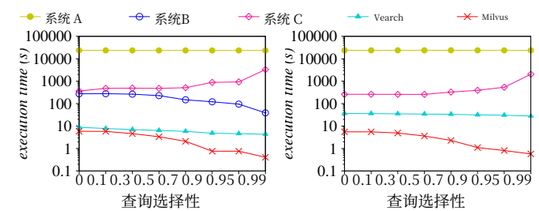
ACKNOWLEDGMENTS

Milvus is a multi-year project that involves many engineers at Zilliz. In particular, we thank Shiyu Chen, Qing Li, Yunmei Li, Chenglong Li, Zizhao Chen, Yan Wang, and Yunying Zhang for their contributions. We also thank Haimeng Cai and Chris Warnock for proofreading the paper. Finally, we would like to thank Walid G. Aref and the anonymous reviewers for their valuable feedback.



(a) k = 50 与召回率 >= 0.95 (b) k = 500 与召回率 >= 0.85

图 14：Milvus 中的属性过滤



(a) k = 50 与召回率 >= 0.95 (b) k = 500 与召回率 >= 0.85

图15：属性过滤对比

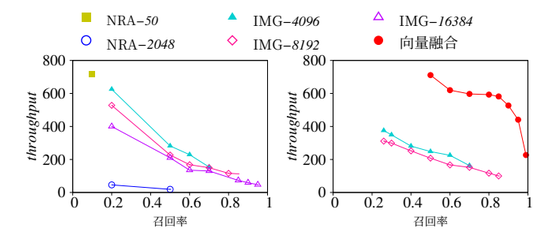
针对每个实体，我们随后使用另一个名为Recipe1M的数据集 [50, 56]，该数据集包含超过一百万道烹饪食谱及对应的食物图像。因此，每个实体均由两个向量描述：文本向量（即食谱描述）和图像向量（即食物图像）。我们从该数据集中随机选取10000个查询，并在本实验中将 k 设为50。本实验采用IVF_FLAT索引构建方法。此外，我们采用加权和作为聚合函数。

图16a展示了以欧氏距离为相似性度量的结果。我们对比了不同 k （50和2048）下的标准NRA算法与我们提出的迭代合并算法（简称为"IMG"）在不同 k' （4096、8192和16384）下的表现。结果表明，标准NRA方法要么执行速度慢，要么召回率低。具体而言，NRA-50方法虽快，但召回率仅为0.1；NRA-2048虽略微提升了召回率（最高达0.5），但性能仍较低；而我们的迭代合并算法（当 k' 为4096时）在召回率相近的情况下，比NRA-2048快 15× 倍。这是因为IMG无需每次均调用向量查询处理，且其堆结构的维护开销更低。

图16b展示了内积度量下的结果。我们将迭代合并（IMG-4096和IMG-8192）与向量融合进行了对比。结果表明，向量融合快了3.4× ~ 5.8× 倍，因为它仅需执行一次top-k 向量相似性搜索。

8 相关工作

向量相似性搜索（又称高维最近邻搜索）是一个被广泛研究的课题，既涵盖近似搜索（例如， [7, 41]），也涵盖精确搜索（例如， [38, 42]）。本工作聚焦于近似搜索，以实现高性能。以往关于近似搜索的研究大致可分为四类：基于局部敏感哈希的 [23, 23, 24, 32, 40, 44, 45, 48, 73]、基于树的 [17, 46, 54, 57]、基于图的 [20, 43, 49, 61, 72]，以及基于量化的 [3, 6, 22, 27, 33, 35]。然而，这些研究均仅关注索引本身，



(a) 欧氏距离 (b) 内积

图16：Milvus中的多向量处理

而 Milvus 是一个功能完备的向量数据管理系统，包含索引、查询引擎、GPU引擎、存储引擎及分布式系统。此外，Milvus 具有可扩展的索引框架，可轻松集成上述各类索引，也可按需引入任何新型索引。目前还存在一些面向向量相似性搜索的开源库，例如 Faiss [35] 和 SPTAG [14]；但它们属于库而非完整系统。我们在表 1 中总结了这些差异。

近期一些工业级向量数据管理系统（例如Alibaba PASE [68] 和阿里巴巴 AnalyticDB-V [65]）并未针对向量进行专门优化。其方法是将关系型数据库扩展以支持向量。因此，如实验所示，其性能严重受损。而Vearch [39] 等专用向量系统则不适用于十亿级数据，且Vearch的性能显著低于Milvus。

此外，还存在基于GPU的向量搜索引擎，例如 [35, 72]。其中，[72] 针对GPU优化了HNSW算法，但其假设数据规模足够小，能够完全装入GPU内存；Faiss [35] 也支持GPU，但当数据无法全部装入GPU内存时，它会按需加载整个数据段，从而导致性能下降。相比之下，Milvus开发了一种全新的混合索引（SQ8H），在不进行动态数据加载的前提下，融合了GPU与CPU各自的优势，从而实现高效的查询处理。

本工作契合构建专用数据引擎的发展趋势，因为"一刀切"的方案并不适用于所有场景 [60]，，例如专用图引擎 [18]，、物联网引擎 [21]，、时间序列数据库 [55]，以及科学数据库 [59]。在此背景下，Milvus正是一种专用于管理向量数据的专用数据引擎。

9 结论

在本工作中，我们分享了近年来在Zilliz公司构建Milvus系统的实践经验。Milvus已被数百家企业采用，目前是LF AI & Data 基金会孵化阶段的项目。展望未来，我们计划利用FPGA加速Milvus。我们已在FPGA上实现了IVF_PQ索引构建，初步结果令人鼓舞。另一个既有趣又具挑战性的方向是将Milvus架构设计为云原生数据管理系统，目前我们正积极推进此项工作。

致谢

Milvus 是一个历时多年的项目，凝聚了 Zilliz 多位工程师的心血。特别感谢陈世宇、李青、李韵梅、李成隆、陈子钊、王艳和张韵颖所做出的贡献。我们还要感谢蔡海萌和 Chris Warnock 对本文进行校对。最后，衷心感谢 Walid G. Aref 及匿名审稿人提出的宝贵意见。

REFERENCES

[1] 2020. *Annoy: Approximate Nearest Neighbors Oh Yeah.* https://github.com/spotify/annoy
[2] 2020. *ElasticSearch: Open Source, Distributed, RESTful Search Engine.* https://github.com/elastic/elasticsearch
[3] 2020. *Facebook Faiss.* https://github.com/facebookresearch/faiss
[4] 2020. *Vearch: A Distributed System for Embedding-based Retrieval.* https://github.com/vearch/vearch
[5] Reza Akbarinia, Esther Pacitti, and Patrick Valduriez. 2007. Best Position Algorithms for Top-k Queries. In *International Conference on Very Large Data Bases (VLDB).* 495–506.
[6] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proceedings of the VLDB Endowment (PVLDB)* 9, 4 (2015), 288–299.
[7] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2018. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Computing Research Repository (CoRR)* abs/1807.05614 (2018).
[8] Artem Babenko and Victor S. Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 2055–2063.
[9] Dávid Bajusz, Anita Rácz, and Károly Héberger. 2015. Why Is Tanimoto Index An Appropriate Choice For Fingerprint-Based Similarity Calculations? *Journal of Cheminformatics* 7 (2015).
[10] Tadas Baltrusaitis, Chaitanya Ahuja, and Louis-Philippe Morency. 2019. Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 41, 2 (2019), 423–443.
[11] Oren Barkan and Noam Koenigstein. 2016. ITEM2VEC: Neural Item Embedding for Collaborative Filtering. In *IEEE International Workshop on Machine Learning for Signal Processing (MLSP).* 1–6.
[12] Kaushik Chakrabarti, Surajit Chaudhuri, and Venkatesh Ganti. 2011. Interval-based Pruning for Top-k Processing over Compressed Lists. In *International Conference on Data Engineering (ICDE).* 709–720.
[13] Hongming Chen, Ola Engkvist, Yinhai Wang, Marcus Olivecrona, and Thomas Blaschke. 2018. The Rise of Deep Learning in Drug Discovery. *Drug Discovery Today* 23, 6 (2018), 1241–1250.
[14] Qi Chen, Haidong Wang, Mingqin Li, Gang Ren, Scarlett Li, Jeffery Zhu, Jason Li, Chuanjie Liu, Lintao Zhang, and Jingdong Wang. 2018. *SPTAG: A Library for Fast Approximate Nearest Neighbor Search.* https://github.com/Microsoft/SPTAG
[15] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *ACM Conference on Recommender Systems (RecSys).* 191–198.
[16] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *ACM Conference on Management of Data (SIGMOD).* 215–226.
[17] Sanjoy Dasgupta and Yoav Freund. 2008. Random Projection Trees and Low Dimensional Manifolds. In *ACM Symposium on Theory of Computing (STOC).* 537–546.
[18] Alin Deutsch, Yu Xu, Mingxi Wu, and Victor E. Lee. 2020. Aggregation Support for Modern Graph Analytics in TigerGraph. In *ACM Conference on Management of Data (SIGMOD).* 377–392.
[19] Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal Aggregation Algorithms for Middleware. In *ACM Symposium on Principles of Database Systems (PODS).* 102–113.
[20] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment (PVLDB)* 12, 5 (2019), 461–474.
[21] Christian Garcia-Arellano, Adam J. Storm, David Kalmuk, Hamdi Roumani, Ronald Barber, Yuanyuan Tian, Richard Sidle, Fatma Özcan, Matt Spilchen, Josh Tiefenbach, Daniel C. Zilio, Lan Pham, Kostas Rakopoulos, Alexander Cheung, Darren Pepper, Imran Sayyid, Gidon Gershinsky, Gal Lushi, and Hamid Pirahesh. 2020. Db2 Event Store: A Purpose-Built IoT Database Engine. *Proceedings of the VLDB Endowment (PVLDB)* 13, 12 (2020), 3299–3312.
[22] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 36, 4 (2014), 744–755.
[23] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *International Conference on Very Large Data Bases (VLDB).* 518–529.
[24] Long Gong, Huayi Wang, Mitsunori Ogihara, and Jun Xu. 2020. iDEC: Indexable Distance Estimating Codes for Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment (PVLDB)* 13, 9 (2020), 1483–1497.
[25] Mihajlo Grbovic and Haibin Cheng. 2018. Real-time Personalization using Embeddings for Search Ranking at Airbnb. In *ACM Conference on Knowledge Discovery & Data Mining (KDD).* 311–320.
[26] Martin Grohe. 2020. Word2vec, Node2vec, Graph2vec, X2vec: Towards a Theory of Vector Embeddings of Structured Data. In *ACM Symposium on Principles of Database Systems (PODS).* 1–16.
[27] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *International Conference on Machine Learning (ICML).*
[28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 770–778.
[29] D. Frank Hsu and Isak Taksa. 2005. Comparing Rank and Score Combination Methods for Data Fusion in Information Retrieval. *Information Retrieval (IR)* 8, 3 (2005), 449–480.
[30] Tongwen Huang, Zhiqi Zhang, and Junlin Zhang. 2019. FiBiNET: Combining Feature Importance and Bilinear Feature Interaction for Click-through Rate Prediction. In *ACM Conference on Recommender Systems (RecSys).* 169–177.
[31] Ihab F. Ilyas, George Beskales, and Mohamed A. Soliman. 2008. A Survey of Top-*k* Query Processing Techniques in Relational Database Systems. *ACM Computing Surveys (CSUR)* 40, 4 (2008), 11:1–11:58.
[32] Omid Jafari, Parth Nagarkar, and Jonathan Montaño. 2020. mmLSH: A Practical and Efficient Technique for Processing Approximate Nearest Neighbor Queries on Multimedia Data. *Computing Research Repository (CoRR)* abs/2003.06415 (2020).
[33] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 33, 1 (2011), 117–128.
[34] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in One Billion Vectors: Re-rank with Source Coding. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP).* 861–864.
[35] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* (2019).
[36] Timothy King. 2019. *80 Percent of Your Data Will Be Unstructured in Five Years.* https://solutionsreview.com/data-management/80-percent-of-your-data-will-be-unstructured-in-five-years/
[37] Quoc V. Le and Tomas Mikolov. 2014. Distributed Representations of Sentences and Documents. In *International Conference on Machine Learning (ICML).* 1188–1196.
[38] Hui Li, Tsz Nam Chan, Man Lung Yiu, and Nikos Mamoulis. 2017. FEXIPRO: Fast and Exact Inner Product Retrieval in Recommender Systems. In *ACM Conference on Management of Data (SIGMOD).* 835–850.
[39] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-Commerce Platform. In *Middleware.* 9–16.
[40] Mingjie Li, Ying Zhang, Yifang Sun, Wei Wang, Ivor W. Tsang, and Xuemin Lin. 2020. I/O Efficient Approximate Nearest Neighbour Search based on Learned Functions. In *International Conference on Data Engineering (ICDE).* 289–300.
[41] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data - Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 32, 8 (2020), 1475–1488.
[42] Yuliang Li, Jianguo Wang, Benjamin Pullman, Nuno Bandeira, and Yannis Papakonstantinou. 2019. Index-Based, High-Dimensional, Cosine Threshold Querying with Optimality Guarantees. In *International Conference on Database Theory (ICDT).* 11:1–11:20.
[43] Peng-Cheng Lin and Wan-Lei Zhao. 2019. A Comparative Study on Hierarchical Navigable Small World Graphs. *Computing Research Repository (CoRR)* abs/1904.02077 (2019).
[44] Wanqi Liu, Hanchen Wang, Ying Zhang, Wei Wang, and Lu Qin. 2019. I-LSH: I/O Efficient c-Approximate Nearest Neighbor Search in High-Dimensional Space. In *International Conference on Data Engineering (ICDE).* 1670–1673.
[45] Kejing Lu and Mineichi Kudo. 2020. R2LSH: A Nearest Neighbor Search Scheme Based on Two-dimensional Projected Spaces. In *International Conference on Data Engineering (ICDE).* 1045–1056.
[46] Kejing Lu, Hongya Wang, Wei Wang, and Mineichi Kudo. 2020. VHP: Approximate Nearest Neighbor Search via Virtual Hypersphere Partitioning. *Proceedings of the VLDB Endowment (PVLDB)* 13, 9 (2020), 1443–1455.
[47] Chen Luo and Michael J. Carey. 2020. LSM-based Storage Techniques: A Survey. *VLDB Journal* 29, 1 (2020), 393–418.
[48] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2017. Intelligent Probing for Locality Sensitive Hashing: Multi-Probe LSH and Beyond. *Proceedings of the VLDB Endowment (PVLDB)* 10, 12 (2017), 2021–2024.
[49] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 42, 4 (2020), 824–836.
[50] Javier Marín, Aritro Biswas, Ferda Ofli, Nicholas Hynes, Amaia Salvador, Yusuf Aytar, Ingmar Weber, and Antonio Torralba. 2018. Recipe1M: A Dataset for Learning Cross-Modal Embeddings for Cooking Recipes and Food Images. *Computing Research Repository (CoRR)* abs/1810.06553 (2018).
[51] Adam C. Mater and Michelle L. Coote. 2019. Deep Learning in Chemistry. *Journal of Chemical Information and Modeling* 59, 6 (2019), 2545–2559.
[52] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. In *International Conference*

参考文献

[1] 2020. *Annoy: Approximate Nearest Neighbors Oh Yeah.* https://github.com/spotify/annoy
[2] 2020. *ElasticSearch: Open Source, Distributed, RESTful Search Engine.* https://github.com/elastic/elasticsearch
[3] 2020. *Facebook Faiss.* https://github.com/facebookresearch/faiss
[4] 2020. *Vearch: A Distributed System for Embedding-based Retrieval.* https://github.com/vearch/vearch
[5] Reza Akbarinia, Esther Pacitti, and Patrick Valduriez. 2007. Best Position Algorithms for Top-k Queries. In *International Conference on Very Large Data Bases (VLDB).* 495–506.
[6] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proceedings of the VLDB Endowment (PVLDB)* 9, 4 (2015), 288–299.
[7] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2018. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Computing Research Repository (CoRR)* abs/1807.05614 (2018).
[8] Artem Babenko and Victor S. Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 2055–2063.
[9] Dávid Bajusz, Anita Rácz, and Károly Héberger. 2015. Why Is Tanimoto Index An Appropriate Choice For Fingerprint-Based Similarity Calculations? *Journal of Cheminformatics* 7 (2015).
[10] Tadas Baltrusaitis, Chaitanya Ahuja, and Louis-Philippe Morency. 2019. Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 41, 2 (2019), 423–443.
[11] Oren Barkan and Noam Koenigstein. 2016. ITEM2VEC: Neural Item Embedding for Collaborative Filtering. In *IEEE International Workshop on Machine Learning for Signal Processing (MLSP).* 1–6.
[12] Kaushik Chakrabarti, Surajit Chaudhuri, and Venkatesh Ganti. 2011. Interval-based Pruning for Top-k Processing over Compressed Lists. In *International Conference on Data Engineering (ICDE).* 709–720.
[13] Hongming Chen, Ola Engkvist, Yinhai Wang, Marcus Olivecrona, and Thomas Blaschke. 2018. The Rise of Deep Learning in Drug Discovery. *Drug Discovery Today* 23, 6 (2018), 1241–1250.
[14] Qi Chen, Haidong Wang, Mingqin Li, Gang Ren, Scarlett Li, Jeffery Zhu, Jason Li, Chuanjie Liu, Lintao Zhang, and Jingdong Wang. 2018. *SPTAG: A Library for Fast Approximate Nearest Neighbor Search.* https://github.com/Microsoft/SPTAG
[15] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *ACM Conference on Recommender Systems (RecSys).* 191–198.
[16] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *ACM Conference on Management of Data (SIGMOD).* 215–226.
[17] Sanjoy Dasgupta and Yoav Freund. 2008. Random Projection Trees and Low Dimensional Manifolds. In *ACM Symposium on Theory of Computing (STOC).* 537–546.
[18] Alin Deutsch, Yu Xu, Mingxi Wu, and Victor E. Lee. 2020. Aggregation Support for Modern Graph Analytics in TigerGraph. In *ACM Conference on Management of Data (SIGMOD).* 377–392.
[19] Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal Aggregation Algorithms for Middleware. In *ACM Symposium on Principles of Database Systems (PODS).* 102–113.
[20] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment (PVLDB)* 12, 5 (2019), 461–474.
[21] Christian Garcia-Arellano, Adam J. Storm, David Kalmuk, Hamdi Roumani, Ronald Barber, Yuanyuan Tian, Richard Sidle, Fatma Özcan, Matt Spilchen, Josh Tiefenbach, Daniel C. Zilio, Lan Pham, Kostas Rakopoulos, Alexander Cheung, Darren Pepper, Imran Sayyid, Gidon Gershinsky, Gal Lushi, and Hamid Pirahesh. 2020. Db2 Event Store: A Purpose-Built IoT Database Engine. *Proceedings of the VLDB Endowment (PVLDB)* 13, 12 (2020), 3299–3312.
[22] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 36, 4 (2014), 744–755.
[23] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *International Conference on Very Large Data Bases (VLDB).* 518–529.
[24] Long Gong, Huayi Wang, Mitsunori Ogihara, and Jun Xu. 2020. iDEC: Indexable Distance Estimating Codes for Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment (PVLDB)* 13, 9 (2020), 1483–1497.
[25] Mihajlo Grbovic and Haibin Cheng. 2018. Real-time Personalization using Embeddings for Search Ranking at Airbnb. In *ACM Conference on Knowledge Discovery & Data Mining (KDD).* 311–320.
[26] Martin Grohe. 2020. Word2vec, Node2vec, Graph2vec, X2vec: Towards a Theory of Vector Embeddings of Structured Data. In *ACM Symposium on Principles of Database Systems (PODS).* 1–16.
[27] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *International Conference on Machine Learning (ICML).*
[28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 770–778.
[29] D. Frank Hsu and Isak Taksa. 2005. Comparing Rank and Score Combination Methods for Data Fusion in Information Retrieval. *Information Retrieval (IR)* 8, 3 (2005), 449–480.
[30] Tongwen Huang, Zhiqi Zhang, and Junlin Zhang. 2019. FiBiNET: Combining Feature Importance and Bilinear Feature Interaction for Click-through Rate Prediction. In *ACM Conference on Recommender Systems (RecSys).* 169–177.
[31] Ihab F. Ilyas, George Beskales, and Mohamed A. Soliman. 2008. A Survey of Top-*k* Query Processing Techniques in Relational Database Systems. *ACM Computing Surveys (CSUR)* 40, 4 (2008), 11:1–11:58.
[32] Omid Jafari, Parth Nagarkar, and Jonathan Montaño. 2020. mmLSH: A Practical and Efficient Technique for Processing Approximate Nearest Neighbor Queries on Multimedia Data. *Computing Research Repository (CoRR)* abs/2003.06415 (2020).
[33] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 33, 1 (2011), 117–128.
[34] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in One Billion Vectors: Re-rank with Source Coding. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP).* 861–864.
[35] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* (2019).
[36] Timothy King. 2019. *80 Percent of Your Data Will Be Unstructured in Five Years.* https://solutionsreview.com/data-management/80-percent-of-your-data-will-be-unstructured-in-five-years/
[37] Quoc V. Le and Tomas Mikolov. 2014. Distributed Representations of Sentences and Documents. In *International Conference on Machine Learning (ICML).* 1188–1196.
[38] Hui Li, Tsz Nam Chan, Man Lung Yiu, and Nikos Mamoulis. 2017. FEXIPRO: Fast and Exact Inner Product Retrieval in Recommender Systems. In *ACM Conference on Management of Data (SIGMOD).* 835–850.
[39] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-Commerce Platform. In *Middleware.* 9–16.
[40] Mingjie Li, Ying Zhang, Yifang Sun, Wei Wang, Ivor W. Tsang, and Xuemin Lin. 2020. I/O Efficient Approximate Nearest Neighbour Search based on Learned Functions. In *International Conference on Data Engineering (ICDE).* 289–300.
[41] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data - Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 32, 8 (2020), 1475–1488.
[42] Yuliang Li, Jianguo Wang, Benjamin Pullman, Nuno Bandeira, and Yannis Papakonstantinou. 2019. Index-Based, High-Dimensional, Cosine Threshold Querying with Optimality Guarantees. In *International Conference on Database Theory (ICDT).* 11:1–11:20.
[43] Peng-Cheng Lin and Wan-Lei Zhao. 2019. A Comparative Study on Hierarchical Navigable Small World Graphs. *Computing Research Repository (CoRR)* abs/1904.02077 (2019).
[44] Wanqi Liu, Hanchen Wang, Ying Zhang, Wei Wang, and Lu Qin. 2019. I-LSH: I/O Efficient c-Approximate Nearest Neighbor Search in High-Dimensional Space. In *International Conference on Data Engineering (ICDE).* 1670–1673.
[45] Kejing Lu and Mineichi Kudo. 2020. R2LSH: A Nearest Neighbor Search Scheme Based on Two-dimensional Projected Spaces. In *International Conference on Data Engineering (ICDE).* 1045–1056.
[46] Kejing Lu, Hongya Wang, Wei Wang, and Mineichi Kudo. 2020. VHP: Approximate Nearest Neighbor Search via Virtual Hypersphere Partitioning. *Proceedings of the VLDB Endowment (PVLDB)* 13, 9 (2020), 1443–1455.
[47] Chen Luo and Michael J. Carey. 2020. LSM-based Storage Techniques: A Survey. *VLDB Journal* 29, 1 (2020), 393–418.
[48] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2017. Intelligent Probing for Locality Sensitive Hashing: Multi-Probe LSH and Beyond. *Proceedings of the VLDB Endowment (PVLDB)* 10, 12 (2017), 2021–2024.
[49] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 42, 4 (2020), 824–836.
[50] Javier Marín, Aritro Biswas, Ferda Ofli, Nicholas Hynes, Amaia Salvador, Yusuf Aytar, Ingmar Weber, and Antonio Torralba. 2018. Recipe1M: A Dataset for Learning Cross-Modal Embeddings for Cooking Recipes and Food Images. *Computing Research Repository (CoRR)* abs/1810.06553 (2018).
[51] Adam C. Mater and Michelle L. Coote. 2019. Deep Learning in Chemistry. *Journal of Chemical Information and Modeling* 59, 6 (2019), 2545–2559.
[52] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. In *International Conference*

on Learning Representations (ICLR).

[53] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. In *Annual Conference on Neural Information Processing Systems (NeurIPS).* 3111–3119.

[54] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 36, 11 (2014), 2227–2240.

[55] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. 2015. Gorilla: A Fast, Scalable, in-Memory Time Series Database. *Proceedings of the VLDB Endowment (PVLDB)* 8, 12 (2015), 1816–1827.

[56] Amaia Salvador, Nicholas Hynes, Yusuf Aytar, Javier Marin, Ferda Ofli, Ingmar Weber, and Antonio Torralba. 2017. Learning Cross-modal Embeddings for Cooking Recipes and Food Images. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 3068–3076.

[57] Chanop Silpa-Anan and Richard I. Hartley. 2008. Optimised KD-trees for Fast Image Descriptor Matching. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 1–8.

[58] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *International Conference on Learning Representations (ICLR).*

[59] Michael Stonebraker, Anastasia Ailamaki, Jeremy Kepner, and Alexander S. Szalay. 2012. The Future of Scientific Data Bases. In *International Conference on Data Engineering (ICDE).* 7–8.

[60] Michael Stonebraker and Ugur Çetintemel. 2005. "One Size Fits All": An Idea Whose Time Has Come and Gone (Abstract). In *International Conference on Data Engineering (ICDE).* 2–11.

[61] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Rand-NSG: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Annual Conference on Neural Information Processing Systems (NeurIPS).* 13748–13758.

[62] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2020. Optimal Join Algorithms Meet Top-k. In *ACM Conference on Management of Data (SIGMOD).* 2659–2665.

[63] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *ACM Conference on Management of Data (SIGMOD).* 1041–1052.

[64] Jianguo Wang, Chunbin Lin, Yannis Papakonstantinou, and Steven Swanson. 2017. An Experimental Study of Bitmap Compression vs. Inverted List Compression. In *ACM Conference on Management of Data (SIGMOD).* 1041–1052.

[65] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proceedings of the VLDB Endowment (PVLDB)* 13, 12 (2020), 3152–3165.

[66] Peter Willett. 2014. The Calculation of Molecular Structural Similarity: Principles and Practice. *Molecular Informatics* 33, 6–7 (2014), 403–413.

[67] Susan Wojcicki. 2020. *YouTube at 15: My Personal Journey and the Road Ahead.* https://blog.youtube/news-and-events/youtube-at-15-my-personal-journey

[68] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *ACM Conference on Management of Data (SIGMOD).* 2241–2253.

[69] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *ACM Conference on Knowledge Discovery & Data Mining (KDD).* 974–983.

[70] Shaoting Zhang, Ming Yang, Timothée Cour, Kai Yu, and Dimitris N. Metaxas. 2015. Query Specific Rank Fusion for Image Retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 37, 4 (2015), 803–815.

[71] Shilin Zhang and Hangbin Yu. 2018. Person Re-Identification by Multi-Camera Networks for Internet of Things in Smart Cities. *IEEE Access* 6 (2018), 76111–76117.

[72] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *International Conference on Data Engineering (ICDE).* 1033–1044.

[73] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. *Proceedings of the VLDB Endowment (PVLDB)* 13, 5 (2020), 643–655.

SIGMOD ' 21，2021年6月20–25日，中国虚拟会议 Wang 等

on Learning Representations (ICLR).

[53] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. In *Annual Conference on Neural Information Processing Systems (NeurIPS).* 3111–3119.

[54] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 36, 11 (2014), 2227–2240.

[55] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. 2015. Gorilla: A Fast, Scalable, in-Memory Time Series Database. *Proceedings of the VLDB Endowment (PVLDB)* 8, 12 (2015), 1816–1827.

[56] Amaia Salvador, Nicholas Hynes, Yusuf Aytar, Javier Marin, Ferda Ofli, Ingmar Weber, and Antonio Torralba. 2017. Learning Cross-modal Embeddings for Cooking Recipes and Food Images. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 3068–3076.

[57] Chanop Silpa-Anan and Richard I. Hartley. 2008. Optimised KD-trees for Fast Image Descriptor Matching. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* 1–8.

[58] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *International Conference on Learning Representations (ICLR).*

[59] Michael Stonebraker, Anastasia Ailamaki, Jeremy Kepner, and Alexander S. Szalay. 2012. The Future of Scientific Data Bases. In *International Conference on Data Engineering (ICDE).* 7–8.

[60] Michael Stonebraker and Ugur Çetintemel. 2005. "One Size Fits All": An Idea Whose Time Has Come and Gone (Abstract). In *International Conference on Data Engineering (ICDE).* 2–11.

[61] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Rand-NSG: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Annual Conference on Neural Information Processing Systems (NeurIPS).* 13748–13758.

[62] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2020. Optimal Join Algorithms Meet Top-k. In *ACM Conference on Management of Data (SIGMOD).* 2659–2665.

[63] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *ACM Conference on Management of Data (SIGMOD).* 1041–1052.

[64] Jianguo Wang, Chunbin Lin, Yannis Papakonstantinou, and Steven Swanson. 2017. An Experimental Study of Bitmap Compression vs. Inverted List Compression. In *ACM Conference on Management of Data (SIGMOD).* 1041–1052.

[65] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proceedings of the VLDB Endowment (PVLDB)* 13, 12 (2020), 3152–3165.

[66] Peter Willett. 2014. The Calculation of Molecular Structural Similarity: Principles and Practice. *Molecular Informatics* 33, 6–7 (2014), 403–413.

[67] Susan Wojcicki. 2020. *YouTube at 15: My Personal Journey and the Road Ahead.* https://blog.youtube/news-and-events/youtube-at-15-my-personal-journey

[68] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *ACM Conference on Management of Data (SIGMOD).* 2241–2253.

[69] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *ACM Conference on Knowledge Discovery & Data Mining (KDD).* 974–983.

[70] Shaoting Zhang, Ming Yang, Timothée Cour, Kai Yu, and Dimitris N. Metaxas. 2015. Query Specific Rank Fusion for Image Retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 37, 4 (2015), 803–815.

[71] Shilin Zhang and Hangbin Yu. 2018. Person Re-Identification by Multi-Camera Networks for Internet of Things in Smart Cities. *IEEE Access* 6 (2018), 76111–76117.

[72] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *International Conference on Data Engineering (ICDE).* 1033–1044.

[73] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. *Proceedings of the VLDB Endowment (PVLDB)* 13, 5 (2020), 643–655.